

# Unit-1 : Introduction of Internet of Things (IoT)

## 1. Introduction to IoT

- IoT (Internet of Things) एक ऐसी technology है जो physical objects (Things) को internet से connect करके data को collect, exchange और process करने की सुविधा देती है।
- इसके द्वारा devices के बीच communication होता है और बिना human intervention के smart decision लिए जाते हैं।
- IoT का use smart home, smart city, healthcare, agriculture, industry automation आदि में किया जाता है।

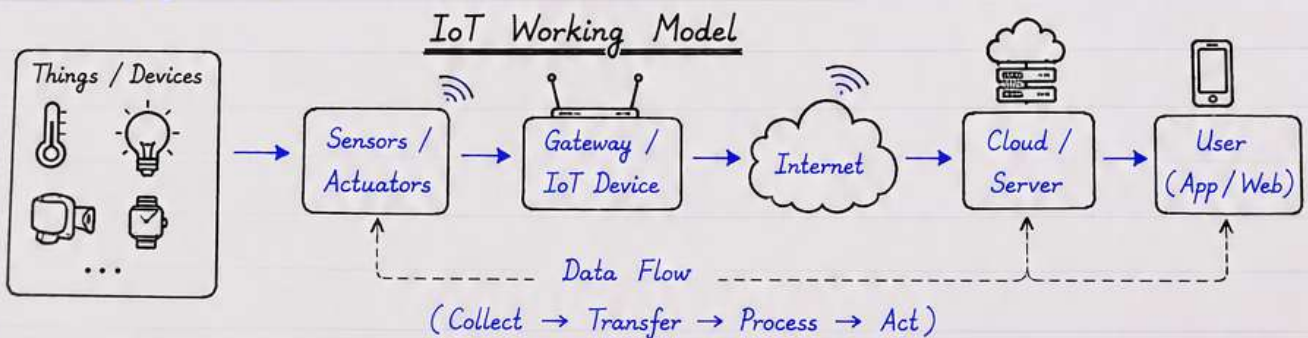
## 2. Defining IoT

IoT को simple शब्दों में इस प्रकार define किया जा सकता है :

IoT एक network of physical objects (Things) है, जिनमें sensors, software और other technologies embedded होती हैं, ताकि वे internet के माध्यम से data को collect, exchange और act कर सकें।

## 3. Things in IoT

- "Things" से तात्पर्य उन physical objects या devices से है जो environment से data collect करते हैं और उसे internet के माध्यम से भेजते या प्राप्त करते हैं।
- ये devices smart होते हैं और इनमें sensing, processing, communication की क्षमता होती है।
- Examples : Temperature sensor, Smart watch, CCTV camera, Smart bulb, Refrigerator, Industrial machines आदि।



## 4. Key Characteristics of IoT

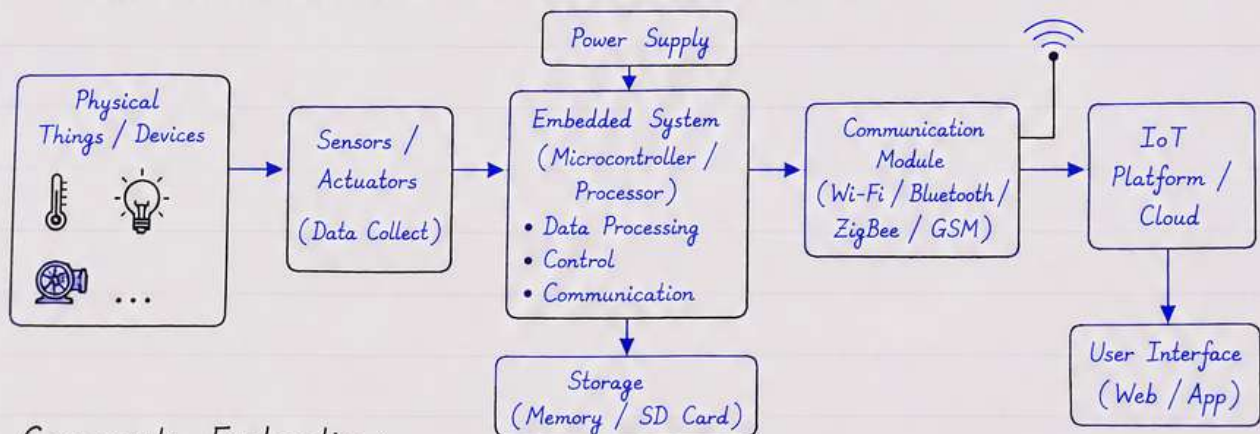
| S.No. | Characteristic | Description                                         |
|-------|----------------|-----------------------------------------------------|
| 1.    | Connectivity   | Things are connected to internet and to each other. |
| 2.    | Sensing        | Sensors collect data from environment.              |
| 3.    | Data Exchange  | Data is transmitted and received over network.      |
| 4.    | Intelligence   | Data is processed and smart decisions are taken.    |
| 5.    | Automation     | Actions are taken automatically without human help. |

Note : IoT का मुख्य उद्देश्य real-time monitoring, efficient management, resource saving और smart life को enable करना है।

## 1. Characteristics of IoT (IoT की विशेषताएँ)

- ① **Connectivity (कनेक्टिविटी)** : IoT devices internet के माध्यम से एक-दूसरे और system से जुड़ते हैं।
- ② **Unique Identification (अनूठी पहचान)** : हर device / thing की एक unique ID होती है ताकि उसे पहचान कर communicate किया जा सके।
- ③ **Sensing (सेंसिंग)** : IoT devices sensors के द्वारा environment से data collect करते हैं जैसे temperature, light, motion आदि।
- ④ **Intelligence (बुद्धिमत्ता)** : Collected data को processing करके smart decision लिए जाते हैं।
- ⑤ **Heterogeneity (विविधता)** : IoT में अलग-अलग प्रकार के devices, networks और platforms एक साथ काम करते हैं।
- ⑥ **Dynamic & Self-configuring (गतिशीलता)** : Devices अपने आप configure हो सकते हैं और environment के अनुसार काम करते हैं।
- ⑦ **Scalability (विस्तार क्षमता)** : IoT system को आसानी से expand किया जा सकता है और नए devices add किए जा सकते हैं।
- ⑧ **Security (सुरक्षा)** : Data और devices को unauthorized access, hacking और attack से protect करना आवश्यक होता है।

## 2. Physical Design of IoT (IoT का भौतिक डिजाइन)



### Components Explanation :

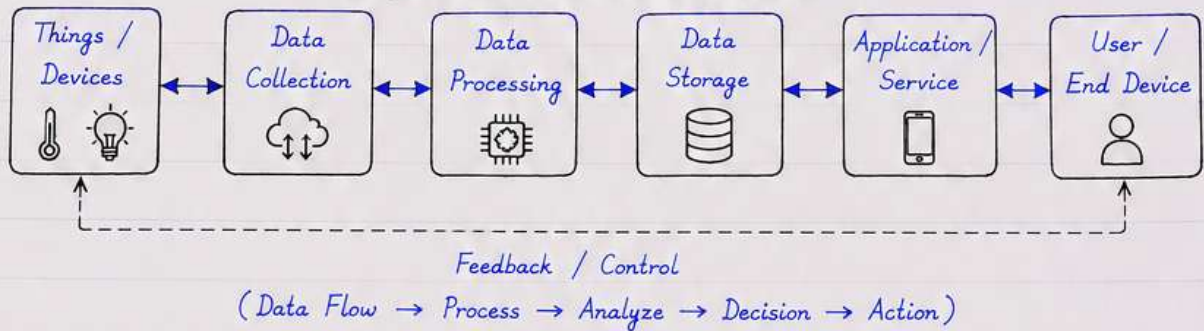
- **Things / Devices** : Real world objects जिन पर sensors और actuators लगे होते हैं।
- **Sensors / Actuators** : Sensors environment से data collect करते हैं और actuators किसी action को perform करते हैं।
- **Embedded System** : Microcontroller / processor data को process करता है और control signals generate करता है।
- **Communication Module** : यह module data को internet / network के माध्यम से cloud या अन्य devices तक भेजता है।
- **Storage** : Data को temporary या permanent store करने के लिए memory का उपयोग होता है।
- **IoT Platform / Cloud** : Data को store, analyze और manage किया जाता है।
- **User Interface** : User web या mobile app के द्वारा system को monitor और control करता है।

**Note :** Physical design का उद्देश्य physical world और digital world को connect करना है ताकि data collect, process और act किया जा सके।

## 1. Logical Design of IoT (IoT का तार्किक डिजाइन)

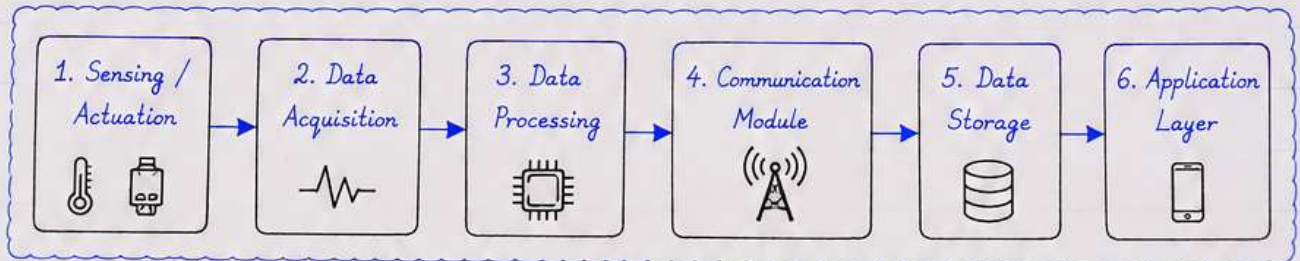
- Logical design IoT system का conceptual view होता है जिसमें data कैसे flow करेगा, कौन-कौन से modules interact करेंगे और system कैसे काम करेगा, यह define किया जाता है।
- इसमें hardware की जगह functions, data और communication पर focus किया जाता है।
- Logical design system के behaviour को describe करता है।
- यह design software architecture, data model और communication model के आधार पर बनाया जाता है।

### Logical Architecture of IoT



## 2. Functional Blocks of IoT (IoT के क्रियात्मक खंड)

IoT system को मुख्य रूप से निम्न functional blocks में divide किया जाता है -



1. Sensing / Actuation : Sensors environment से data collect करते हैं और actuators किसी action को perform करते हैं।
2. Data Acquisition : Sensors से प्राप्त data को capture करके digital form में convert किया जाता है।
3. Data Processing : Collected data को process, filter और analyze किया जाता है ताकि useful information प्राप्त हो सके।
4. Communication Module : यह module data को internet / network के माध्यम से दूसरे devices या cloud तक भेजता है।
5. Data Storage : Processed data को temporary या permanent storage (Cloud, Server, Database) में store किया जाता है।
6. Application Layer : User द्वारा mobile app, web dashboard या software के माध्यम से data को visualize, control और manage किया जाता है।

**Note :** Logical design system के flow, functions और interaction को plan करता है, जबकि physical design hardware components और connections को define करता है।

# IoT Protocols (IoT प्रोटोकॉल)

- IoT में devices और systems के बीच data communication के लिए protocols का use किया जाता है।
- ये protocols ensure करते हैं कि data secure, reliable और efficient तरीके से send और receive हो सके।

## 1. IoT Protocols क्या होते हैं ?

IoT Protocols वे rules और standards होते हैं जिनके द्वारा IoT devices एक-दूसरे और network के साथ communicate करते हैं।

## 2. IoT Protocols की आवश्यकता

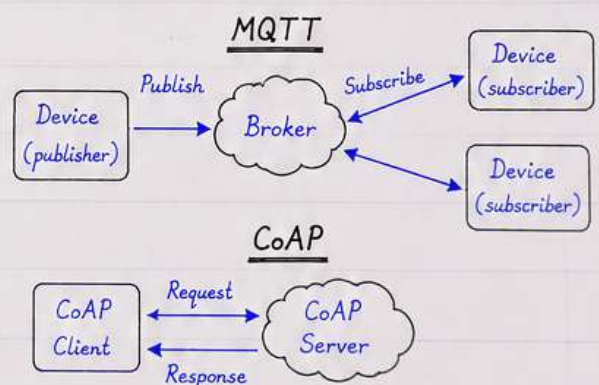
- Low power devices के लिए efficient communication provide करना।
- Limited bandwidth में data transfer करना।
- Reliable और secure data exchange सुनिश्चित करना।
- Different devices और platforms के बीच interoperability provide करना।

## 3. Popular IoT Protocols

| Protocol   | Layer             | Use                               | Features                                        | Transport        |
|------------|-------------------|-----------------------------------|-------------------------------------------------|------------------|
| MQTT       | Application Layer | Publish / Subscribe Messaging     | Lightweight, Low bandwidth, Low power           | TCP              |
| CoAP       | Application Layer | Constrained devices communication | RESTful, Lightweight, UDP based                 | UDP              |
| HTTP/HTTPS | Application Layer | Web based data access             | Widely used, Secure (HTTPS), Request / Response | TCP              |
| AMQP       | Application Layer | Enterprise message queuing        | Reliable, Secure, Supports queuing              | TCP              |
| DDS        | Middleware Layer  | Real-time data exchange           | High performance, Scalable, Real-time           | TCP / UDP        |
| LwM2M      | Application Layer | Device management & monitoring    | Lightweight, Secure, Device management          | UDP / SMS / CoAP |

## 4. Protocol Comparision (MQTT vs CoAP)

| MQTT                                                                                                                                                                                       | CoAP                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Publish / Subscribe model</li><li>• Runs on TCP</li><li>• Low bandwidth</li><li>• Requires broker</li><li>• Best for unreliable networks</li></ul> | <ul style="list-style-type: none"><li>• Request / Response model</li><li>• Runs on UDP</li><li>• Very lightweight</li><li>• No broker required</li><li>• Best for constrained devices</li></ul> |



## 5. Key Points

- MQTT और CoAP IoT में सबसे ज्यादा use होने वाले protocols हैं।
- MQTT long term connections के लिए अच्छा है जबकि CoAP constrained devices के लिए बेहतर है।
- Security के लिए protocols में encryption (TLS/DTLS) का use किया जाता है।

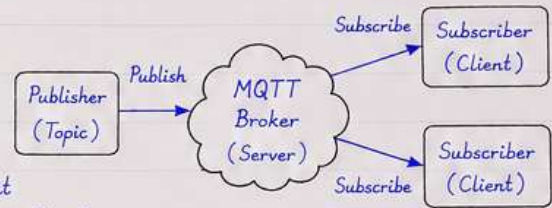
**Note :** सही protocol का चयन application की requirement, device capability और network condition पर निर्भर करता है।

# IoT Protocols : MQTT, CoAP, AMQP, DDS

IoT में communication के लिए अलग-अलग protocols का use किया जाता है।  
हर protocol की अपनी architecture, features और use case होते हैं।

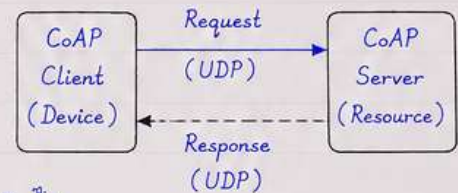
## 1. MQTT (Message Queuing Telemetry Transport)

- Type : Lightweight messaging protocol
- Layer : Application Layer
- Transport: TCP
- Design : Publish / Subscribe model
- Working : MQTT में एक central server (Broker) होता है। Client (Publisher) messages को topic पर publish करता है और जो client उस topic को subscribe करता है, उसे message प्राप्त होता है।
- Features : Lightweight, Low bandwidth, Reliable, QoS level (0,1,2)
- Use : Remote monitoring, Smart home, Sensor data collection



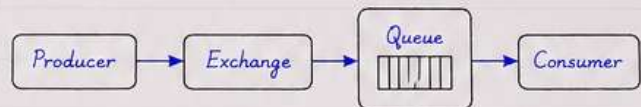
## 2. CoAP (Constrained Application Protocol)

- Type : Lightweight RESTful protocol
- Layer : Application Layer
- Transport : UDP
- Design : Client / Server model
- Working : CoAP HTTP जैसा ही काम करता है लेकिन यह UDP पर चलता है। यह constrained devices के लिए बनाया गया है, जहाँ low power और low bandwidth होते हैं।
- Features : Lightweight, Low overhead, Confirmable / Non-confirmable messages
- Use : Smart devices, Wearables, Home automation



## 3. AMQP (Advanced Message Queuing Protocol)

- Type : Message-oriented middleware protocol
- Layer : Application Layer
- Transport : TCP
- Design : Message Queue model
- Working : AMQP में producer messages को exchange के द्वारा queue में भेजता है और consumer queue से message प्राप्त करता है।
- Features : Reliable, Secure, Routing, Acknowledgement, Queuing
- Use : Enterprise applications, Banking, E-commerce, Payment systems



## 4. DDS (Data Distribution Service)

- Type : Real-time publish / subscribe protocol
- Layer : Middleware Layer (RTPS)
- Transport : UDP
- Design : Data-Centric Publish / Subscribe
- Working : DDS data को topics के रूप में publish करता है और subscribers को real-time में data मिलती है। यह high performance और real-time systems के लिए बनाया गया है।
- Features : Real-time, Scalable, QoS configurable, Decentralized
- Use : Industrial automation, Robotics, Autonomous vehicles, Healthcare



Comparison Table

| Protocol | Transport | Model                  | Best For               | Specialty          |
|----------|-----------|------------------------|------------------------|--------------------|
| MQTT     | TCP       | Publish / Subscribe    | Low bandwidth devices  | Very Lightweight   |
| CoAP     | UDP       | Client / Server (REST) | Constrained devices    | Low overhead       |
| AMQP     | TCP       | Message Queue          | Enterprise systems     | Reliable Messaging |
| DDS      | UDP       | Data-Centric Pub/Sub   | Real-time applications | High performance   |

### Note :

सभी protocols का चुनाव application requirement, device capability, network condition और real-time need के अनुसार किया जाता है।

# IoT Communication Models (IoT संचार मॉडल)

- IoT devices और systems के बीच data exchange और communication के लिए अलग-अलग communication models का use किया जाता है।
- इन models के आधार पर data flow, control, scalability और resource utilization को manage किया जाता है।

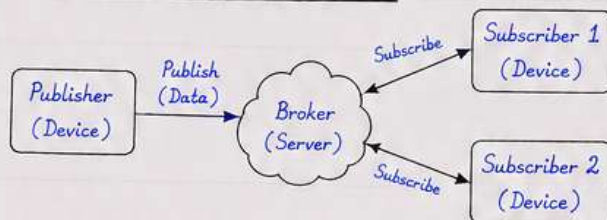
## 1. Request - Response Model



Use Case : Web based IoT applications, Configuration, On-demand data.

- Client किसी resource के लिए request भेजता है।
- Server उस request को process करके response भेजता है।
- यह model HTTP/REST (CoAP) में use होता है।
- यह simple और easy to implement होता है।

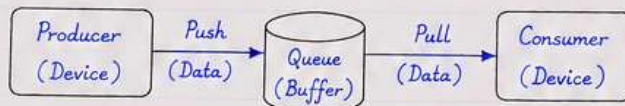
## 2. Publish - Subscribe Model



Use Case : Real-time monitoring, Sensor data streaming, Smart home, Industrial automation.

- Publisher data को किसी topic पर publish करता है।
- Broker उस data को उन सभी subscribers तक भेजता है जिन्होंने उस topic को subscribe किया है।
- One-to-Many communication support करता है।
- यह model MQTT में use होता है।

## 3. Push - Pull Model



Use Case : Enterprise applications, Message queuing, Task distribution.

- Producer data को queue/buffer में push करता है।
- Consumer जब ready होता है तब queue से data pull करके process करता है।
- यह model AMQP में commonly use होता है।
- Reliable delivery और decoupling provide करता है।

## 4. Peer to Peer Model



Use Case : Device control, Local connectivity, Wearables, Smart vehicles.

- Devices आपस में सीधे communicate करते हैं, किसी central server की आवश्यकता नहीं होती।
- यह model low latency और high speed communication के लिए उपयोगी होता है।
- यह model CoAP observe / BLE / Wi-Fi Direct में देखा जाता है।

## Comparison of IoT Communication Models

| Model             | Pattern      | Scalability | Latency  | Use Case                            | Example Protocols               |
|-------------------|--------------|-------------|----------|-------------------------------------|---------------------------------|
| Request-Response  | 1-to-1       | Low         | Medium   | Web apps, Configuration             | HTTP, CoAP                      |
| Publish-Subscribe | 1-to-Many    | High        | Low      | Real-time monitoring                | MQTT                            |
| Push-Pull         | Many-to-Many | High        | Medium   | Messaging, Task queue               | AMQP                            |
| Peer to Peer      | Many-to-Many | Medium      | Very Low | Local control, Device communication | CoAP Observe, BLE, Wi-Fi Direct |

Note : सही communication model का चयन application requirement, network condition, energy consumption और scalability को ध्यान में रखकर किया जाता है।

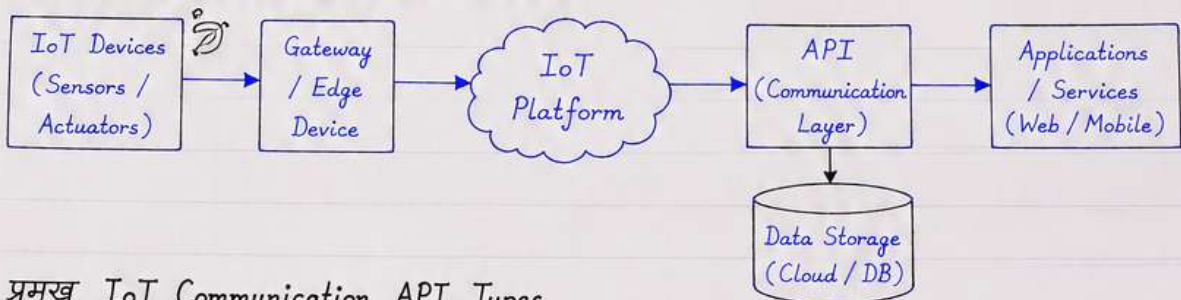
# IoT Communication API's (Application Programming Interfaces)

- API (Application Programming Interface) एक set of rules और protocols होता है जो विभिन्न software applications को एक-दूसरे के साथ communicate करने की अनुमति देता है।
- IoT में API's का use devices, platforms और cloud services के बीच data exchange और control के लिए किया जाता है।

## 1. IoT Communication API की आवश्यकता

- Different devices और platforms के बीच interoperability प्रदान करता है।
- Data को easily access, manage और share करने में मदद करता है।
- Scalable और secure communication enable करता है।
- Application development को fast और efficient बनाता है।

## 2. IoT Communication API Architecture



## 3. प्रमुख IoT Communication API Types

| API Type      | Description                                                                                  | Use Case                                     | Example                                  |
|---------------|----------------------------------------------------------------------------------------------|----------------------------------------------|------------------------------------------|
| RESTful API   | HTTP/HTTPS पर आधारित stateless API, JSON या XML data exchange के लिए।                        | Web और cloud based IoT applications के लिए।  | ThingSpeak API, AWS IoT REST API         |
| WebSocket API | Full-duplex communication provide करता है, जो real-time data transfer के लिए उपयोगी होता है। | Real-time monitoring, live updates के लिए।   | Socket-IO, Firebase Realtime API         |
| MQTT API      | Lightweight publish/subscribe protocol जो low bandwidth और low power devices के लिए best है। | Remote monitoring, sensor data streaming     | Eclipse Paho API, Mosquitto MQTT API     |
| CoAP API      | Constrained devices के लिए designed lightweight protocol (REST like)।                        | Smart home, Wearables, low power IoT devices | Californium CoAP API                     |
| AMQP API      | Reliable message queuing और routing के लिए use होता है।                                      | Enterprise level IoT systems, Task queue     | RabbitMQ HTTP API, Azure Service Bus API |
| DDS API       | Real-time, high performance और scalable communication के लिए।                                | Industrial automation, Healthcare, Robotics  | RTI Connnext DDS API                     |

## 4. IoT API Workflow



## 5. Benefits of Using API's in IoT

- Standardization के कारण development time कम होता है।
- Security mechanisms (Authentication, Authorization, Encryption) आसानी से integrate होते हैं।
- Third-party services और applications को easily integrate किया जा सकता है।
- System की scalability और maintainability improve होती है।

**Note :** IoT API's का सही चयन application requirement, device capability, security और data nature पर निर्भर करता है।

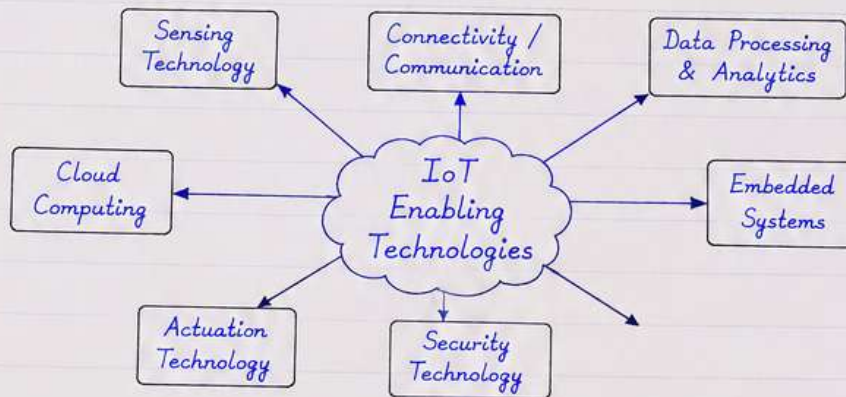
# IoT Enabling Technologies (IoT को सक्षम बनाने वाली तकनीकें)

- IoT कई आधुनिक तकनीकों के एक साथ उपयोग से संभव होता है। ये सभी तकनीकें मिलकर IoT system को smart, connected और efficient बनाती हैं।
- इन technologies के बिना IoT devices data collect, communicate, process और act नहीं कर सकते।

## 1. IoT Enabling Technologies - Overview

IoT को सक्षम बनाने वाली प्रमुख तकनीकें निम्नलिखित हैं -

- Sensing Technology
- Connectivity / Communication Technology
- Data Processing & Analytics
- Cloud Computing
- Embedded Systems
- Actuation Technology
- Security Technology



### 1. Sensing Technology

- Sensors physical world से data collect करते हैं जैसे temperature, humidity, light, motion आदि।
- यह real-time data प्राप्त करने के लिए उपयोगी होती हैं।
- Examples: Temperature sensor, PIR sensor, Gas sensor, GPS आदि।

### 2. Connectivity / Communication Technology

- यह technology devices और systems के बीच data transfer को संभव बनाती है।
- Wired और Wireless दोनों माध्यमों का उपयोग होता है।
- Examples: Wi-Fi, Bluetooth, ZigBee, LoRa, Cellular (4G/5G), NB-IoT आदि।

### 3. Data Processing & Analytics

- Collect किए गए data को process, filter और analyze किया जाता है ताकि useful information प्राप्त हो सके।
- यह decision making और automation के लिए बहुत महत्वपूर्ण होती है।
- Tools: Big Data, Machine Learning, AI, Data Mining आदि।

### 7. Security Technology

- IoT में data और devices की security बहुत जरूरी है क्योंकि यह cyber attacks से protect करती है।
- यह authentication, authorization, encryption और secure communication सुनिश्चित करती है।
- Examples: SSL/TLS, AES, Firewall, VPN, Token based authentication आदि।

### 4. Cloud Computing

- Cloud IoT data को store, manage और process करने के लिए scalable platform प्रदान करता है।
- यह remote access, backup और cost efficiency भी प्रदान करता है।
- Examples: AWS IoT, Microsoft Azure IoT, Google Cloud IoT आदि।

### 5. Embedded Systems

- IoT devices में छोटे embedded systems होते हैं जो sensing, processing और control का कार्य करते हैं।
- ये low power, compact और specific tasks के लिए design होते हैं।
- Examples: Arduino, Raspberry Pi, ESP32, ARM based boards आदि।

### 6. Actuation Technology

- Actuators sensors से प्राप्त data के आधार पर physical action perform करते हैं।
- यह system को automatic और responsive बनाती है।
- Examples: Relay, Motor, Solenoid, LED, Buzzer आदि।

Note : इन सभी technologies का integration ही IoT को प्रभावी, सुरक्षित और विश्वसनीय बनाता है। यह technologies मिलकर smart home, smart city, industrial automation, healthcare, agriculture आदि कई क्षेत्रों में IoT applications को संभव बनाती हैं।

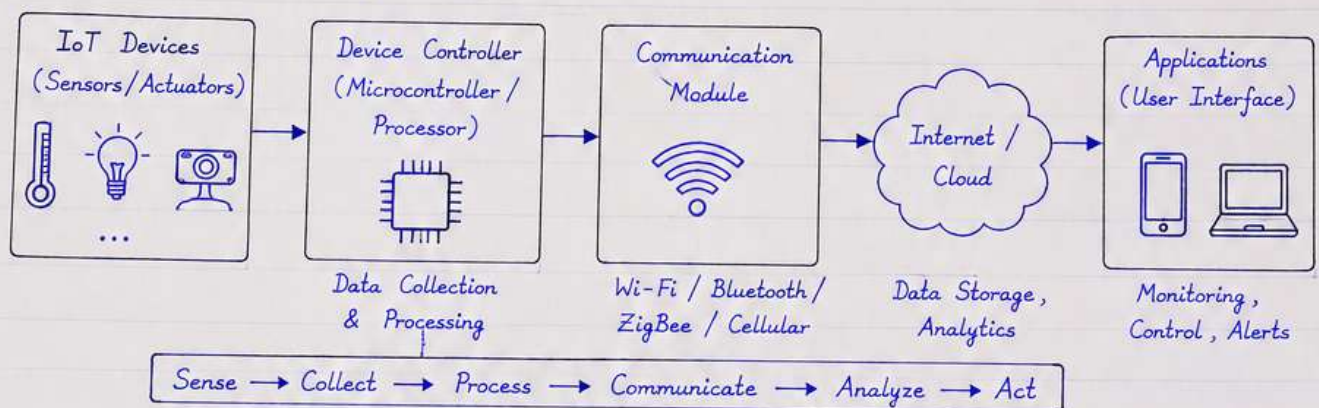
## Unit-2 : IoT Devices

### Topic : How electronic devices fit with the Internet of Things, and Why they are Important

\* **Introduction** : IoT (Internet of Things) में अनेक electronic devices आपस में connect होकर data collect, share और process करते हैं। ये devices sensors, actuators, microcontrollers और communication modules से मिलकर बने होते हैं जो इंटरनेट के माध्यम से cloud या अन्य devices के साथ data exchange करते हैं।

#### 1. How electronic devices fit with the Internet of Things?

IoT architecture में electronic devices system के different layers में काम करते हैं और data flow को संभव बनाते हैं।



#### 2. IoT Devices के मुख्य घटक (Components)

- **Sensor** : Physical world से data collect करता है (जैसे temperature, light, motion)
- **Actuator** : System को physical action देने के लिए use होता है (जैसे motor, relay)
- **Microcontroller/Processor** : Data को process करता है और device को control करता है।
- **Communication Module** : Internet या network के माध्यम से data भेजता है।
- **Power Supply** : Device को required power प्रदान करता है (battery / adapter / solar)

#### 3. Why they are Important?

- **Real-time Monitoring** : Sensors की मदद से real-time data प्राप्त होता है जिससे सही decision लेना आसान होता है।
- **Automation** : Actuators और controllers के द्वारा manual काम automatic हो जाते हैं, जिससे समय और लागत की बचत होती है।
- **Remote Access & Control** : IoT devices को इंटरनेट के माध्यम से कहीं से भी monitor और control किया जा सकता है।
- **Data-driven Decision** : Collected data को analyze करके बेहतर और smart decision लिए जाते हैं।
- **Efficiency & Productivity** : Processes optimize होते हैं, errors कम होते हैं और productivity बढ़ती है।
- **New Services & Business Opportunities** : IoT devices नए smart solutions और innovative services प्रदान करके business growth में मदद करते हैं।

#### 4. Examples of IoT Devices

| Device Type             | Examples                                          |
|-------------------------|---------------------------------------------------|
| Smart Home Devices      | Smart Thermostat, Smart Light, Smart Door Lock    |
| Wearables               | Fitness Band, Smart Watch, Health Monitor         |
| Industrial IoT Devices  | Smart Sensor, PLC, Industrial Robots              |
| Agriculture IoT Devices | Soil Moisture Sensor, Smart Irrigation System     |
| Healthcare IoT Devices  | Heart Rate Monitor, Smart Bed, Medicine Dispenser |

#### Conclusion :

Electronic devices IoT की नींव हैं। ये devices sensors, processing और communication के माध्यम से real world को digital world से जोड़कर smart और connected environment बनाते हैं।

# Electronic Components : Breadboard and its internal connections

## → Breadboard क्या है?

Breadboard एक reusable solderless prototyping board होता है, जिसका उपयोग electronic circuit को बिना soldering के बनाने और test करने के लिए किया जाता है।

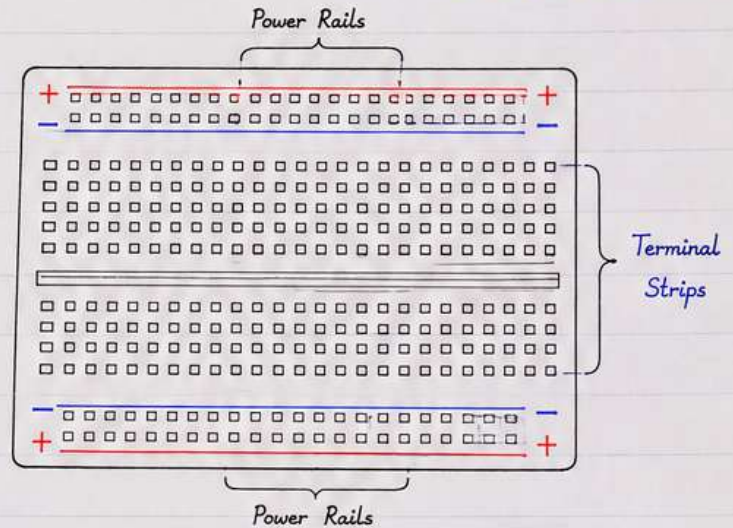
यह components को जल्दी जोड़ने, बदलने और experiment करने के लिए बहुत useful होता है।

### Features :

- Solderless
- Reusable
- Easy to use
- Temporary circuit
- For testing & prototyping

## 1. Breadboard का Structure

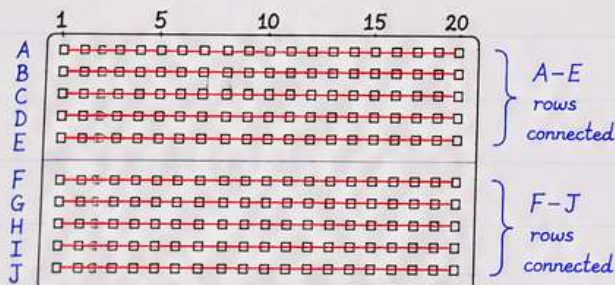
- Breadboard मुख्य रूप से दो भागों में divided होता है :
  - (i) Terminal Strips (R rows और columns वाले holes)
  - (ii) Power Rails (ऊपर और नीचे दिए गए +ve और -ve lines)
- Standard breadboard में 400, 830 या 1660 tie-points होते हैं।



## 2. Breadboard की Internal Connections

### (i) Terminal Strips (Main Area)

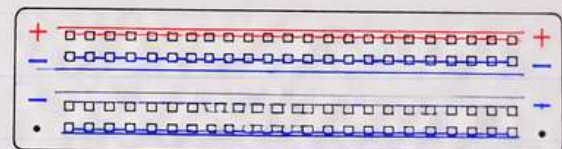
- Breadboard के बीच का हिस्सा rows (A-E और F-J) में divided होता है, और बीच में एक gap होता है।
- हर row में 5 holes आपस में internally connected होते हैं।
- A-E की rows आपस में connected होती हैं, और अलग से F-J की rows आपस में connected होती हैं।
- A-E और F-J के बीच की gap में कोई internal connection नहीं होता।



Note : Column (vertical) में कोई internal connection नहीं होता।

### (ii) Power Rails

- Breadboard के ऊपर और नीचे दिए गए long strips को power rails कहते हैं।
- ऊपर वाली rail में आमतौर पर red line (+ve) और blue line (-ve) होती हैं।
- हर rail के सभी holes एक साथ internally connected होते हैं (horizontal direction में)।
- इन rails का उपयोग power supply देने के लिए किया जाता है।



Top Red line → +ve rail (सब holes connected)  
Top Blue line → -ve rail (सब holes connected)  
Bottom Red line → +ve rail (सब holes connected)  
Bottom Blue line → -ve rail (सब holes connected)

## 3. Breadboard का उपयोग

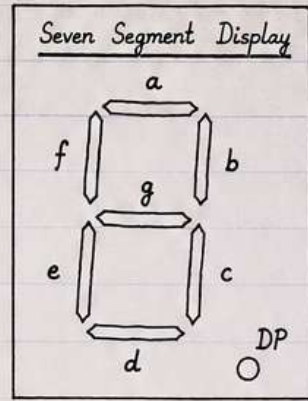
- Circuit को quickly design और test करने के लिए।
- Components को easily जोड़ने और बदलने के लिए।
- Arduino, sensors, ICs और other modules के साथ prototyping करने के लिए।
- Students और engineers के लिए experiment और learning के लिए बहुत useful हैं।

### Tip :

Power rail का उपयोग करते समय ध्यान रखें कि +ve और -ve को सही से connect करें।

# Seven Segment Display on Breadboard

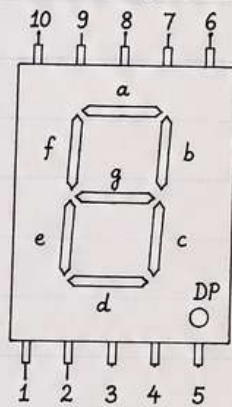
→ **Introduction :** Seven Segment Display एक electronic display होता है जिसमें 7 LEDs (a, b, c, d, e, f, g) होते हैं जो मिलकर 0-9 digits और कुछ alphabets (A, b, C, d, E, F, आदि) को display करते हैं। इसे breadboard पर आसानी से use किया जा सकता है किसी भी microcontroller (Arduino, 8051, PIC, आदि) या logic circuit के साथ।



## 1. Seven Segment Display के Segments

- इसमें 7 segments (a, b, c, d, e, f, g) और एक Decimal Point (DP) होता है।
- Common Type :
  - (i) Common Cathode (CC) - सभी cathode common
  - (ii) Common Anode (CA) - सभी anode common

## 2. Pin Configuration (Common Cathode)



| Pin No. | Segment        | Function      |
|---------|----------------|---------------|
| 1       | e              | Left Bottom   |
| 2       | d              | Bottom        |
| 3       | Common Cathode | Ground (GND)  |
| 4       | c              | Right Bottom  |
| 5       | DP             | Decimal Point |
| 6       | b              | Right Top     |
| 7       | a              | Top           |
| 8       | Common Cathode | Ground (GND)  |
| 9       | f              | Left Top      |
| 10      | g              | Middle        |

## 3. Breadboard Connection (Example)

- यहाँ हम Common Cathode Display का use कर रहे हैं।
- Pin 3 और Pin 8 (Common Cathode) को breadboard के GND से connect करें।
- Segments (a, b, c, d, e, f, g, DP) को current limiting resistors ( $220\Omega - 330\Omega$ ) के through Arduino के digital pins से connect करें।
- जब किसी pin से HIGH (5V) दिया जाता है, तो segment ON हो जाता है। (LOW = OFF)

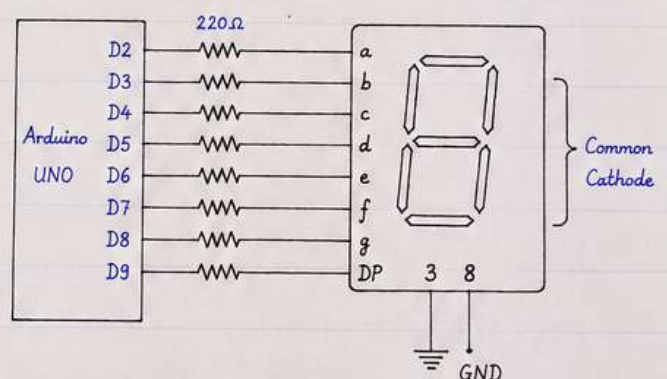
**Note :** अगर आप Common Anode Display use करते हैं तो logic उल्टा होगा : LOW = ON , HIGH = OFF

## 4. Digit Numbers and Segment Pattern (Common Cathode)

| Digit | a | b | c | d | e | f | g | DP |
|-------|---|---|---|---|---|---|---|----|
| 0     | H | H | H | H | H | H | L | L  |
| 1     | L | H | H | L | L | L | L | L  |
| 2     | H | H | L | H | H | L | H | L  |
| 3     | H | H | H | H | L | L | H | L  |
| 4     | L | H | H | L | L | H | H | L  |
| 5     | H | L | H | H | L | H | H | L  |
| 6     | H | L | H | H | H | H | H | L  |
| 7     | H | H | H | L | L | L | L | L  |
| 8     | H | H | H | H | H | H | H | L  |
| 9     | H | H | H | H | L | H | H | L  |

H = HIGH (ON), L = LOW (OFF)

## 5. Circuit Diagram (Example with Arduino UNO)



## 6. Important Points

- हमेशा current limiting resistor का use करें, वरना LED damage हो सकती है।
- Common Cathode में HIGH देने पर segment ON होता है, और Common Anode में LOW देने पर ON होता है।
- Breadboard पर connections हमेशा proper row/column में करें ताकि short circuit न हो।
- Decimal Point (DP) को भी अलग से control कर सकते हैं।

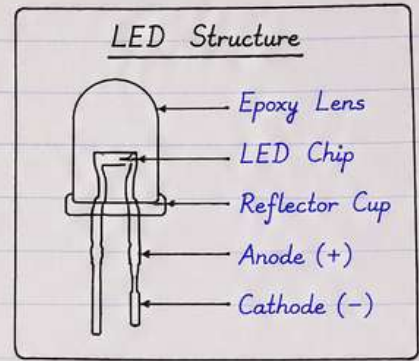
### Applications :

- Digital Clock
- Counters
- Voltage Display
- Score Boards
- Interfacing Projects

# LED and its Connections

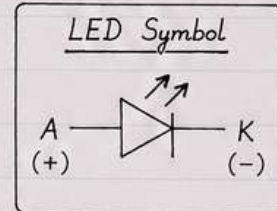
## → Introduction :

LED (Light Emitting Diode) एक semiconductor device होता है, जो forward bias condition में light emit करता है। इसमें कम power consumption होता है, long life होता है और यह digital circuits में indicator या display के लिए उपयोगी होता है।



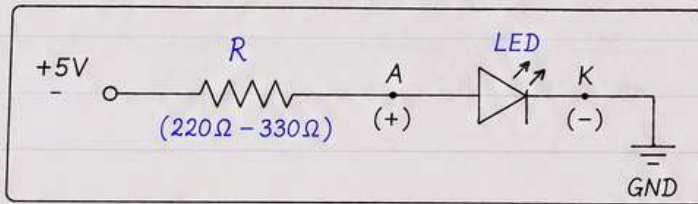
## 1. LED के Terminals

- LED के दो terminals होते हैं - Anode (+) और Cathode (-)
- Longer leg = Anode (+)
- Shorter leg / Flat side = Cathode (-)
- Circuit symbol में Anode बायीं side और Cathode दायीं side होता है।



## 2. LED Connections (Basic)

- LED को हमेशा एक current limiting resistor के साथ series में connect किया जाता है, क्योंकि LED कम current में ही work करता है।



A → Anode (+)

K → Cathode (-)

R → Current Limiting Resistor

## 3. How it Works ?

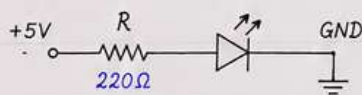
- जब +ve supply Anode की side और -ve supply Cathode की side दी जाती है, तब LED forward biased होता है और light emit करता है।
- यदि connection reverse किया जाए (Anode -ve, Cathode +ve), तो LED glow नहीं करेगा।

### Note :

LED को बिना resistor के सीधे supply से connect नहीं करना चाहिए, वरना LED burn हो सकता है।

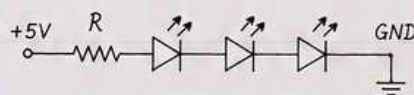
## 4. Types of LED Connections

### (i) Single LED Connection



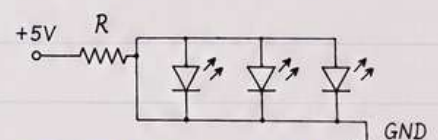
यह सबसे basic connection है, जिसमें एक LED use होता है।

### (ii) Multiple LEDs in Series



इसमें सभी LEDs series में connected होते हैं। Same current सभी में flow करता है।

### (iii) Multiple LEDs in Parallel



इसमें सभी LEDs parallel में connected होते हैं। Same voltage सभी LEDs पर मिलता है।

## 5. Important Points

- LED एक diode की तरह काम करता है।
- Always use current limiting resistor।
- Typical LED forward voltage = 1.8V to 3.3V (color के अनुसार)
- Typical forward current = 5mA to 20mA
- LEDs का उपयोग indicators, displays, sign boards, traffic lights आदि में किया जाता है।

### LED Colors and Typical Forward Voltage

| Color  | Forward Voltage (Approx.) |
|--------|---------------------------|
| Red    | 1.8V - 2.2V               |
| Green  | 2.0V - 3.2V               |
| Blue   | 2.8V - 3.3V               |
| White  | 2.8V - 3.3V               |
| Yellow | 2.0V - 2.2V               |

# 1. Tri-color LED

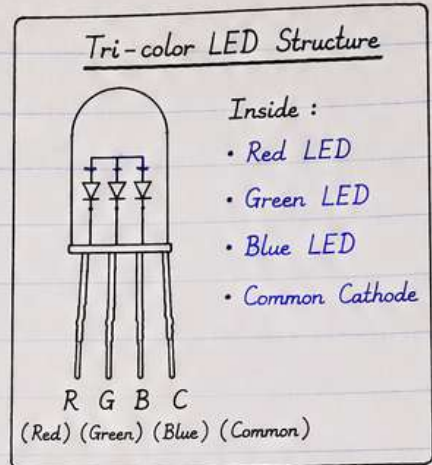
→ Tri-color LED एक special type का LED होता है जिसमें तीन अलग-अलग colors (Red, Green, Blue) एक ही package में होते हैं। इसके चार terminals होते हैं - एक common और तीन individual (R, G, B).

## Tri-color LED Terminals (Common Cathode Type)

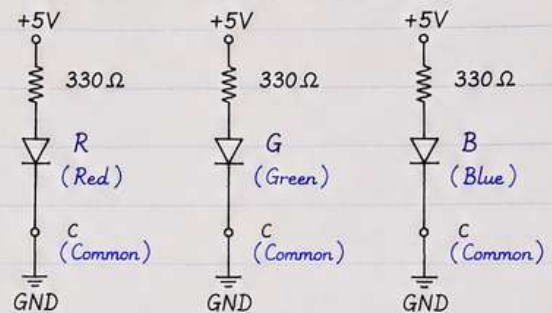
- R - Red LED Anode
- G - Green LED Anode
- B - Blue LED Anode
- C - Common Cathode (-)

## Working :

Common pin (C) को GND से connect किया जाता है। जब किसी भी color pin (R, G या B) को +ve supply (through current limiting resistor) से connect करते हैं, तो उस color की LED glow करती है।



## Tri-color LED Connections (Example)



Note : C (Common) को हमेशा GND से connect करें और R, G, B को +ve supply से resistor लगाकर connect करें।

# 2. Resistor

→ Resistor एक passive electronic component होता है जो current के flow को limit करता है या control करता है। यह voltage drop create करता है और circuits को protect करने में help करता है।

## Resistor का Symbol :



## Resistor का Formula (Ohm's Law) :

$$V = I \times R$$

V = Voltage (Volt)  
I = Current (Ampere)  
R = Resistance (Ohm)

## Types of Resistors :

1. Fixed Resistor (सामान्य Resistor)
2. Variable Resistor (Potentiometer / Rheostat)
3. Special Resistors (LDR, Thermistor, etc.)

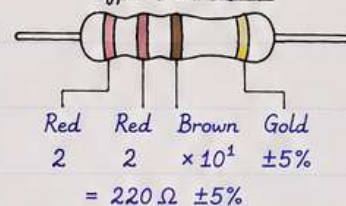
## Uses of Resistor :

- Current को limit करना
- LED और other components को protect करना
- Voltage divider circuits बनाना
- Signal conditioning और biasing के लिए

## Resistor Color Code

| Color      | Digit | Multiplier | Tolerance    |
|------------|-------|------------|--------------|
| Black      | 0     | $10^0$     | -            |
| Brown      | 1     | $10^1$     | $\pm 1\%$    |
| Red        | 2     | $10^2$     | $\pm 2\%$    |
| Orange     | 3     | $10^3$     | -            |
| Yellow     | 4     | $10^4$     | -            |
| Green      | 5     | $10^5$     | $\pm 0.5\%$  |
| Blue       | 6     | $10^6$     | $\pm 0.25\%$ |
| Violet     | 7     | $10^7$     | $\pm 0.10\%$ |
| Grey       | 8     | $10^8$     | $\pm 0.05\%$ |
| White      | 9     | $10^9$     | -            |
| Gold       | -     | $10^{-1}$  | $\pm 5\%$    |
| Silver     | -     | $10^{-2}$  | $\pm 10\%$   |
| (No Color) | -     | -          | $\pm 20\%$   |

## Typical Resistor



## Important Points :

- Resistor की unit Ohm ( $\Omega$ ) होती है।
- $1 \text{ k}\Omega = 1000 \Omega$ ,  $1 \text{ M}\Omega = 1,000,000 \Omega$
- Resistor में current हमेशा high potential से low potential की ओर flow करता है।

# Unit - 3 : IoT Networks

## Topic : Introduction to the components of basic IoT networks

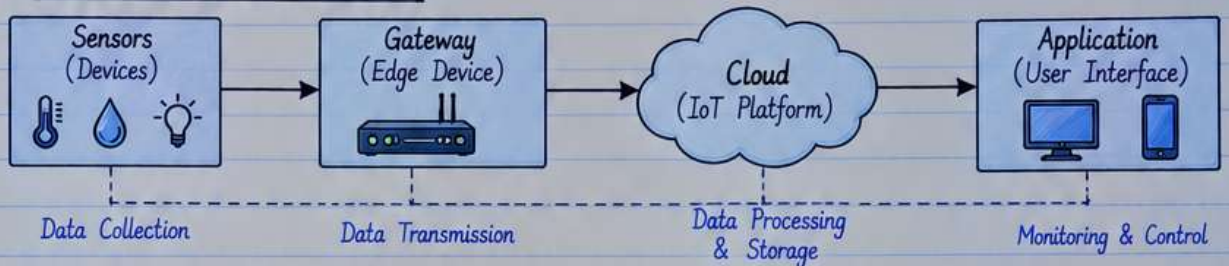
### 1. Introduction :

- IoT (Internet of Things) एक ऐसा Network होता है जिसमें Physical Devices, Sensors, Software और अन्य Technologies इंटरनेट के माध्यम से एक दूसरे से जुड़े होते हैं और Data का आदान-प्रदान करते हैं।
- IoT Networks का मूल उद्देश्य Real-time Data प्राप्त करना, उसे Process करना और आवश्यक कार्य करना होता है।
- एक Basic IoT Network कई Components से मिलकर बना होता है, जो मिलकर Sensing, Communication, Data Processing और Application को संभव बनाते हैं।

### 2. Basic Components of IoT Networks :

- ① IoT Devices (Sensors and Actuators)
- ② Communication Network
- ③ Gateway / Edge Device
- ④ Cloud / IoT Platform
- ⑤ Data Processing and Analytics
- ⑥ Application Layer (User Interface)

### 3. IoT Network Architecture :



### 4. IoT Devices (Sensors and Actuators):

- Sensors Physical Environment से Data Collect करते हैं (जैसे Temperature, Humidity, Light, Motion)।
- Actuators प्राप्त Data के आधार पर Physical Action करते हैं (जैसे Motor को On/Off करना)।
- ये Devices आमतौर पर Microcontroller (जैसे Arduino, Raspberry Pi) के साथ काम करते हैं।

### 5. Communication Network :

- यह Component IoT Devices को Gateway या Cloud से Connect करता है।
- इसमें Wired (Ethernet) और Wireless (Wi-Fi, Bluetooth, Zigbee, LoRa, Cellular) Technologies का उपयोग होता है।
- यह Data को एक स्थान से दूसरे स्थान तक Secure रूप से Transfer करता है।

### 6. Gateway / Edge Device :

- यह Sensors से प्राप्त Data को Collect करता है।
- यह Data को Filter, Process और आवश्यक होने पर Local Level पर Analyze भी करता है।
- इसके बाद यह Data को Cloud Platform पर भेजता है।
- यह अलग-अलग Protocols के बीच Communication में मदद करता है।

### 7. Cloud / IoT Platform :

- यह Data को Store, Manage और Process करने के लिए उपयोग होता है।
- यह बड़ी मात्रा में Data को संभाल सकता है।
- यह Remote Access, Device Management, Security और Data Analytics जैसी सुविधाएँ प्रदान करता है।
- उदाहरण : AWS IoT, Microsoft Azure IoT, Google Cloud IoT, ThingsBoard आदि।

### 8. Data Processing and Analytics :

- यह Component Cloud या Edge पर प्राप्त Data को Process और Analyze करता है।
- Data Analytics की मदद से Useful Information प्राप्त होती है।
- इससे Monitoring, Prediction और Automated Decision लेने में सहायता मिलती है।

### 9. Application Layer (User Interface):

- यह End User के लिए Interface प्रदान करता है।
- इसमें Web Dashboard, Mobile App या Software Application शामिल हो सकते हैं।
- User यहाँ पर Real-time Data देख सकता है, Devices को Control कर सकता है और Alerts प्राप्त कर सकता है।

### 10. Conclusion :

- एक Basic IoT Network के सभी Components मिलकर काम करते हैं ताकि Real-time Data Collection, Communication, Processing और User Interaction संभव हो सके और विभिन्न क्षेत्रों में Smart Solutions प्रदान की जा सकें।

## Topic : The Types of Network Connections

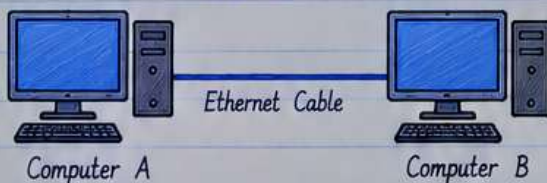
### 1. Introduction :

- Network Connection का अर्थ है दो या दो से अधिक Devices के बीच Data का आदान-प्रदान करने के लिए स्थापित एक माध्यम (Medium)।
- Communication Network में Devices को आपस में जोड़ने के लिए विभिन्न प्रकार की Connection Technologies का उपयोग किया जाता है।
- हर Network Connection का उद्देश्य Data को Reliable, Fast और Secure तरीके से एक स्थान से दूसरे स्थान तक पहुँचाना होता है।

### 2. Types of Network Connections :

#### (i) Wired Connection (Ethernet)

- इसमें Devices को Cable (जैसे Twisted Pair) के द्वारा जोड़ा जाता है।
- यह स्थिर (Stable) और तेज (High Speed) Connection प्रदान करता है।
- यह Office, School, Data Center आदि में अधिक उपयोग होता है।



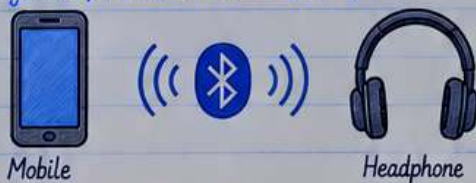
#### (ii) Wireless Connection (Wi-Fi)

- इसमें Devices को बिना Cable के Radio Waves के द्वारा जोड़ा जाता है।
- Wi-Fi एक Popular Wireless Technology है।
- यह Home, Office, Public Places आदि में व्यापक रूप से उपयोग किया जाता है।
- यह Mobility और Flexibility प्रदान करता है।



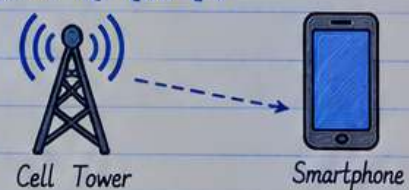
#### (iii) Bluetooth Connection

- यह एक Short Range Wireless Technology है।
- इसका उपयोग कम दूरी (लगभग 10 मीटर) में Devices को जोड़ने के लिए किया जाता है।
- यह Low Power Consumption के लिए जाना जाता है।
- इसका उपयोग Mobile, Headphone, Smartwatch, Keyboard, Mouse आदि में होता है।



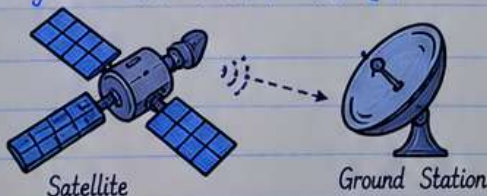
#### (iv) Cellular Network Connection

- यह Mobile Network (2G, 3G, 4G, 5G) के द्वारा Internet Connection प्रदान करता है।
- यह Wide Area Coverage प्रदान करता है।
- इसका उपयोग Smartphones, Tablets, IoT Devices आदि में किया जाता है।
- यह उन स्थानों के लिए उपयोगी है जहाँ Wi-Fi उपलब्ध नहीं होता है।



#### (v) Satellite Connection

- इसमें Communication Satellites के द्वारा Data का आदान-प्रदान किया जाता है।
- यह दूरस्थ और ग्रामीण क्षेत्रों में Internet पहुँचाने के लिए उपयोगी है।
- इसका उपयोग TV Broadcast, Internet Access, Navigation आदि में किया जाता है।



#### (vi) Infrared Connection

- यह एक Short Range Wireless Technology है।
- इसमें Data को Infrared Light के द्वारा Transmit किया जाता है।
- यह बहुत कम दूरी (1-5 मीटर) के लिए उपयोगी है।
- इसका उपयोग TV Remote, Computer, Printer और Mobile Devices में किया जाता है।



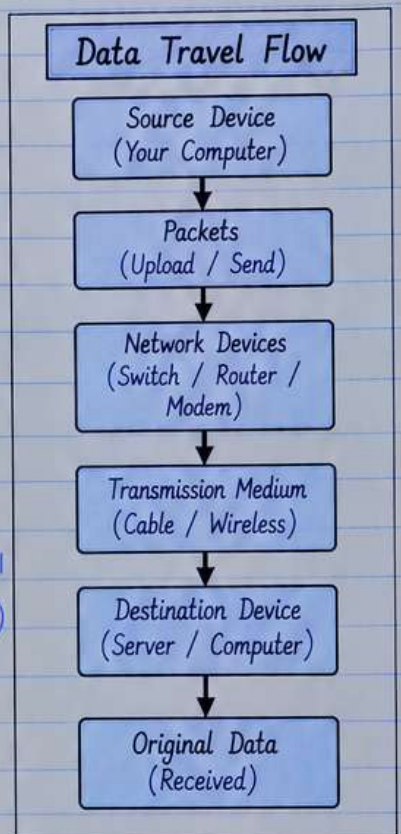
# Topic : How Data Travels Through Them

## 1. Introduction :

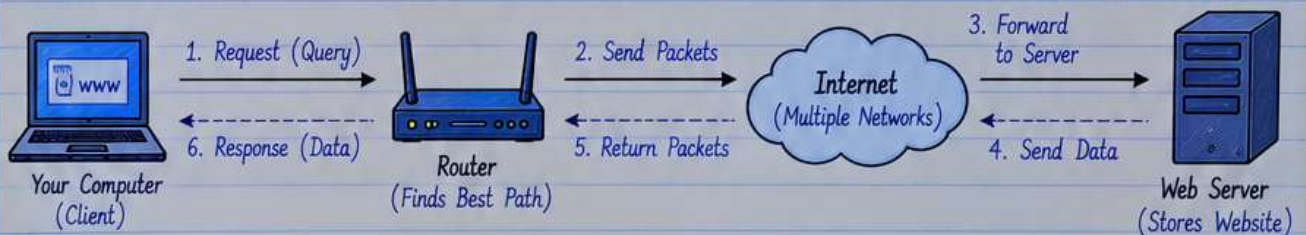
- जब हम Internet या किसी Network का उपयोग करते हैं, तो हमारा Data एक Device से दूसरे Device तक जाता है।
- यह Data अलग-अलग Network Devices (जैसे Switch, Router, Modem, Access Point आदि) से होकर गुजरता है।
- Data छोटे-छोटे भागों (Packets) में विभाजित होकर Travel करता है, ताकि वह सुरक्षित और सही तरीके से अपने Destination तक पहुँच सके।
- हर Device रास्ते में Data को Receive, Process और आगे Forward करता है।

## 2. Data Travel Process (Step by Step) :

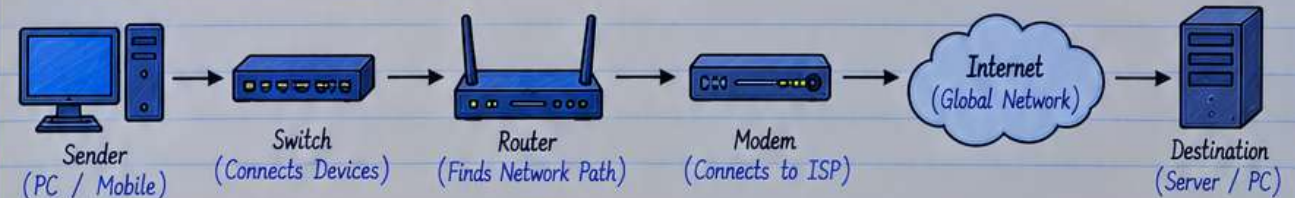
- Data Creation :** User जब कोई Information (जैसे Web Page, Email, Video) भेजता है, तो वह Data के रूप में तैयार होती है।
- Packetization :** यह Data छोटे-छोटे Packets में टूट जाती है, जिनमें Source Address, Destination Address और अन्य Control Information होती है।
- Transmission :** Packets को Physical Medium (Cable या Wireless Signal) के माध्यम से भेजा जाता है।
- Routing :** बीच में आने वाले Devices (Switch, Router आदि) Destination Address को देखकर सही रास्ता चुनते हैं और Packet को आगे भेजते हैं।
- Reception :** Destination Device सभी Packets को प्राप्त करता है।
- Reassembly :** प्राप्त हुए Packets को सही क्रम में जोड़कर Original Data तैयार किया जाता है।



## 3. Real Example : (Web Page Open करना)



## 4. Path of Data Through Different Devices :



## 5. Important Points :

- Data हमेशा Packets के रूप में Travel करता है।
- हर Packet में Source IP, Destination IP और Control Information होती है।
- Different Devices इस Information के आधार पर Packet को आगे भेजते हैं।
- Data अलग-अलग Networks (LAN, WAN, Internet) से होकर जा सकता है।
- यह प्रक्रिया बहुत तेज़ (High Speed) और सुरक्षित (Secure) होती है।

### Key Terms :

- Packet :** Data का छोटा भाग
- Source Address :** भेजने वाले का पता
- Destination Address :** प्राप्त करने वाले का पता
- Routing :** सही रास्ता चुनना
- Transmission Medium :** Data भेजने का माध्यम

# Topic : The role of Internet Protocols

## 1. Introduction :

- Internet Protocols, Internet पर Data Communication के लिए बनाए गए Rules और Standards का समूह हैं।
- ये Protocols विभिन्न Devices के बीच Data को सही तरीके से Send, Receive और Process करने में मदद करते हैं।
- इनका मुख्य उद्देश्य है कि Data अलग-अलग Networks के बीच भी बिना किसी समस्या के अपने Destination तक पहुँच सके।
- Internet Protocols के बिना Internet पर Communication संभव नहीं होता।

## 2. What are Internet Protocols ?

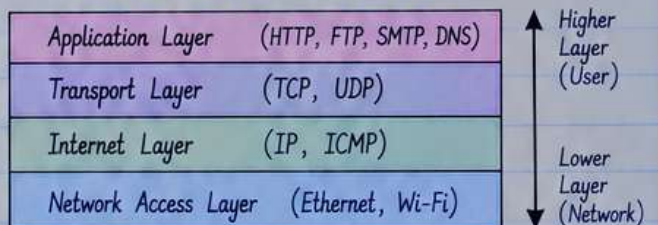
- Internet Protocols वे नियम (Rules) और Procedures हैं जिनके द्वारा Computers और अन्य Network Devices एक-दूसरे के साथ Data का आदान-प्रदान करते हैं।
- ये तय करते हैं कि Data को कैसे Format किया जाएगा, कैसे Address किया जाएगा, कैसे Transmit किया जाएगा और उस Error को कैसे Handle किया जाएगा।
- हर Layer (जैसे Application, Transport, Network, आदि) में अलग-अलग Protocols काम करते हैं।

## 3. Need of Internet Protocols :

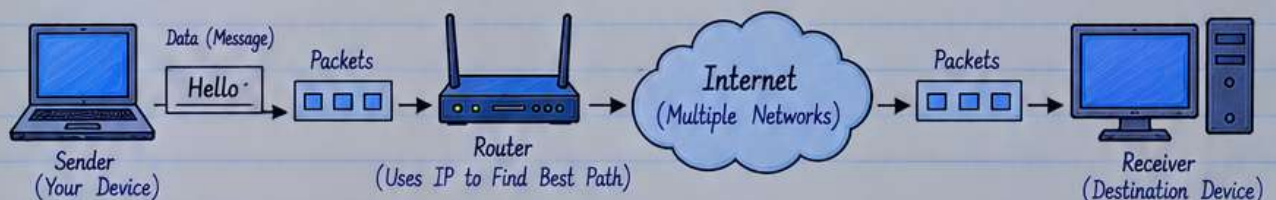
- Different Devices और Operating Systems के बीच Communication के लिए Common Language प्रदान करते हैं।
- Data को सही Address तक पहुँचाने में मदद करते हैं।
- Network के बीच Data Routing और Transfer को संभव बनाते हैं।
- Data की Security, Error Checking और Flow Control में सहायता करते हैं।
- विभिन्न Applications (जैसे Web, Email, File Transfer) को Host करने के लिए आवश्यक होते हैं।

## 4. Protocols and Layered Architecture :

- Internet Protocols को बेहतर Management के लिए Layers में बाँटा गया है (TCP/IP Model)।
- हर Layer में कुछ Specific Protocols काम करते हैं।
- Upper Layer Application से संबंधित कार्य करती हैं और Lower Layer Data को Physical Network के माध्यम से भेजने का कार्य करती हैं।



## 5. How Internet Protocols Work ? (Data Flow)



## 6. Important Internet Protocols and Their Roles :

| Protocol                                 | Layer                | Main Role / Function                                        |
|------------------------------------------|----------------------|-------------------------------------------------------------|
| IP (Internet Protocol)                   | Internet Layer       | Data को Address करना और सही Destination तक पहुँचाना।        |
| TCP (Transmission Control Protocol)      | Transport Layer      | Reliable Data Transfer, Error Control, Flow Control।        |
| UDP (User Datagram Protocol)             | Transport Layer      | Fast Data Transfer (No Guarantee of Delivery)।              |
| HTTP (HyperText Transfer Protocol)       | Application Layer    | Web Pages को Transfer करने के लिए।                          |
| FTP (File Transfer Protocol)             | Application Layer    | Files को Upload और Download करने के लिए।                    |
| SMTP (Simple Mail Transfer Protocol)     | Application Layer    | Email भेजने के लिए।                                         |
| DNS (Domain Name System)                 | Application Layer    | Domain Name को IP Address में बदलने के लिए।                 |
| ICMP (Internet Control Message Protocol) | Internet Layer       | Error Reporting और Network Status Check के लिए (जैसे ping)। |
| Ethernet                                 | Network Access Layer | Local Network में Data Transfer के लिए।                     |
| Wi-Fi (IEEE 802.11)                      | Network Access Layer | Wireless Network में Data Transfer के लिए।                  |

## 7. Example : Web Browsing में Protocols की भूमिका

- जब हम किसी Website खोलते हैं, तो Browser HTTP Protocol के माध्यम से Request भेजता है।
- DNS उस Website के Domain Name को IP Address में बदलता है।
- Data Packets के रूप में Internet के माध्यम से Destination Server तक पहुँचता है (IP द्वारा)।
- TCP यह सुनिश्चित करता है कि सभी Packets सही क्रम में प्राप्त हों।
- Server Response को वापस हमारे Device तक भेजता है और Browser उसे Web Page के रूप में दिखाता है।

## 8. IP Packet Structure (Basic)

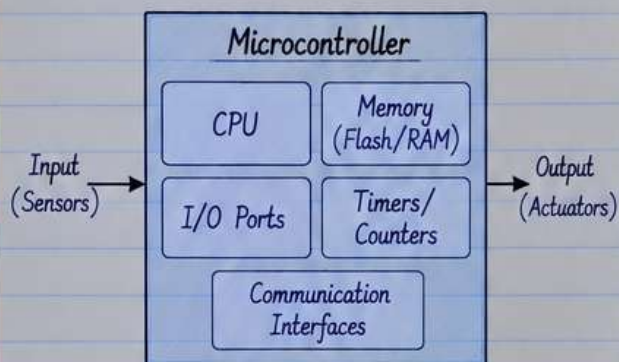
| Version | Header Length | Source IP Address | Destination IP Address | Data (Payload) |
|---------|---------------|-------------------|------------------------|----------------|
|---------|---------------|-------------------|------------------------|----------------|

- Source IP Address : भेजने वाले Device का Address
- Destination IP Address : प्राप्त करने वाले Device का Address
- Data (Payload) : वास्तविक Data (जैसे Web Page, File, आदि)

# Topic : Basic understanding of microcontrollers/Arduino and communication protocols

## 1. What is a Microcontroller ?

- Microcontroller एक छोटा Computer होता है जो एक ही Chip में CPU, Memory, Input/Output Ports और other Control Units को शामिल करता है।
- यह Embedded Systems में उपयोग होता है।
- इसका उपयोग विभिन्न Devices को Control करने के लिए किया जाता है, जैसे Sensors से Data पढ़ना, Actuators को Control करना आदि।
- यह कम Power में काम करता है और Specific Tasks के लिए Design किया जाता है।



## 2. What is Arduino ?

- Arduino एक Open Source Microcontroller Development Board है।
- यह Hardware और Software दोनों का Combination है, जो Beginners के लिए बहुत आसान है।
- Arduino में ATmega series का Microcontroller होता है (जैसे ATmega328P - Arduino Uno में)।
- यह Sensors, LEDs, Motors और अन्य Electronic Devices को आसानी से Interface करने की सुविधा प्रदान करता है।
- Arduino को Programming इसके IDE (Integrated Development Environment) में C/C++ (Arduino Language) का उपयोग करके किया जाता है।

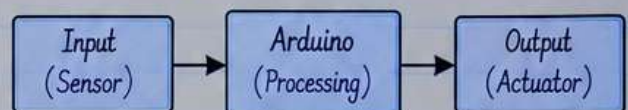


## 3. Microcontroller vs Arduino :

| Feature     | Microcontroller          | Arduino                                  |
|-------------|--------------------------|------------------------------------------|
| Type        | Single Chip              | Development Board (with Microcontroller) |
| Components  | CPU, Memory, I/O etc.    | Microcontroller + Additional Circuitry   |
| Usage       | Used in Embedded Systems | Used for Learning and Prototyping        |
| Programming | Requires Programmer      | Easy (Arduino IDE)                       |
| Cost        | Low                      | Slightly Higher                          |
| Example     | ATmega328, PIC, ARM      | Arduino Uno, Nano, Mega                  |

## 4. Basic Arduino Working :

- ① Write code in Arduino IDE.
- ② Compile and Upload to Arduino board.
- ③ Arduino executes the code.
- ④ It reads input from Sensors.
- ⑤ Processes the data as per program.
- ⑥ Sends output to Actuators (LED, Motor, etc.).



## 5. Introduction to Communication Protocols :

- Communication Protocols उन नियमों का समूह है जो दो Devices के बीच Data के आदान-प्रदान (Send और Receive) को नियंत्रित करते हैं।
- ये Protocols यह तय करते हैं कि Data कैसे Format होगा, कब भेजा जाएगा, कितनी Speed से भेजा जाएगा और Error Handling कैसे होगी।
- IoT और Embedded Systems में Sensors, Modules और अन्य Devices के बीच Communication के लिए ये बहुत महत्वपूर्ण होते हैं।

## 6. Common Communication Protocols :

| Protocol  | Full Form                                   | Key Features / Use                                     |
|-----------|---------------------------------------------|--------------------------------------------------------|
| UART      | Universal Asynchronous Receiver Transmitter | Simple, 2 wires (TX, RX), Short distance communication |
| I2C       | Inter-Integrated Circuit                    | 2 wires (SDA, SCL), Multiple devices, Low speed        |
| SPI       | Serial Peripheral Interface                 | High speed, 4 wires (MOSI, MISO, SCK, CS)              |
| USB       | Universal Serial Bus                        | High speed, used for PC communication                  |
| Wi-Fi     | Wireless Fidelity                           | Wireless communication (Internet)                      |
| Bluetooth | Short Range Wireless                        | Low power, short range wireless communication          |

# Unit 4 : Arduino

## Topic : Arduino device introduction

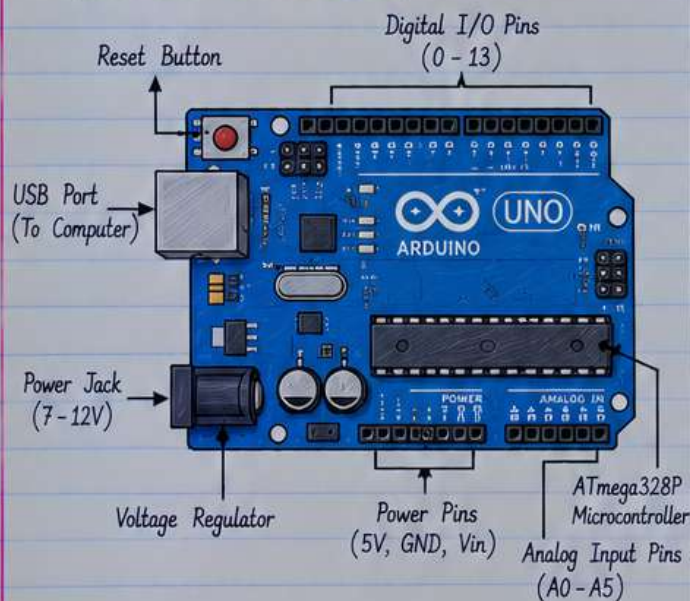
### 1. What is Arduino ?

- Arduino एक Open Source Electronics Platform है जो एक Microcontroller Board, Software (IDE) और Community Support प्रदान करता है।
- इसका उपयोग Sensors, Actuators और अन्य Electronic Devices को Control करने के लिए किया जाता है।
- Arduino का उपयोग Students, Hobbyists और Professionals द्वारा Prototyping, Learning और Real-world Projects के लिए किया जाता है।
- यह Hardware और Software दोनों का Combination है, जिससे बिना ज्यादा Hardware Knowledge के भी Electronics Projects बनाना आसान हो जाता है।

### 2. Features of Arduino :

- Easy to use और Beginner Friendly।
- Open Source Hardware और Software।
- Multiple Digital और Analog Input/Output Pins।
- USB के माध्यम से Computer से Connect होता है।
- C/C++ पर आधारित Arduino IDE में Programming की जाती है।
- Wide range of Boards (Uno, Nano, Mega, etc.)।
- Sensors, Motors, LEDs और विभिन्न Modules के साथ Compatible।
- Low cost, Portable और Versatile।

### 3. Arduino Uno Board :



### 4. Main Components :

| Component                    | Function                                                           |
|------------------------------|--------------------------------------------------------------------|
| Microcontroller (ATmega328P) | Program को Execute करता है और सभी Input/Output को Control करता है। |
| USB Port                     | Computer से Connect होकर Program Upload करने के लिए उपयोग होता है। |
| Power Jack                   | External Power Supply (7-12V) के लिए।                              |
| Digital I/O Pins (0 - 13)    | Digital Input या Output के लिए (जैसे LED, Switch)।                 |
| Analog Input Pins (A0 - A5)  | Analog Sensors (जैसे Temperature, Light) से Data पढ़ने के लिए।     |
| Reset Button                 | Board को Restart करने के लिए।                                      |
| Voltage Regulator            | Input Voltage को Regulate करके Stable 5V प्रदान करता है।           |

### 5. Arduino IDE (Software) :

- Arduino IDE एक Open Source Software है जिसका उपयोग Arduino Board में Program लिखने, Compile करने और Upload करने के लिए किया जाता है।
- यह C और C++ पर आधारित Programming Language का उपयोग करता है।
- इसके द्वारा हम सरल Code लिखकर Sensors से Data पढ़ सकते हैं और Outputs (LED, Motor आदि) को Control कर सकते हैं।
- Arduino IDE में कुछ पहले से बने Examples भी होते हैं जिससे Learning आसान हो जाती है।

### 6. Applications of Arduino :

- Home Automation (जैसे Light, Fan Control)
- Robotics Projects
- IoT (Internet of Things) Devices
- Environmental Monitoring (Temperature, Humidity)
- Security Systems (Motion Sensor, Alarm)
- Automatic Irrigation Systems
- Educational and Research Projects
- Industrial Prototyping

### 7. Different Arduino Boards :

| Board Name       | Key Feature / Use               |
|------------------|---------------------------------|
| Arduino Uno      | Most popular, beginner friendly |
| Arduino Nano     | Small size, compact projects    |
| Arduino Mega     | More I/O pins, complex projects |
| Arduino Mini     | Very small, embedded projects   |
| Arduino Leonardo | Inbuilt USB communication       |

### Basic Steps in Arduino IDE :

- ① Code लिखना (Write)
- ② Verify / Compile करना
- ③ Upload करना (Board में Program भेजना)
- ④ Serial Monitor द्वारा Output देखना

# Topic : Feature of Arduino Device

## 1. Open Source Platform :

- Arduino एक Open Source Hardware और Software Platform हैं।
- इसके Design, Circuit Diagram और Code सभी के लिए freely available होते हैं।
- इसका उपयोग Students, Hobbyists और Professionals द्वारा किया जा सकता है।
- Open Source होने के कारण इसमें Community Support बहुत मजबूत है।

## 3. Affordable and Cost Effective :

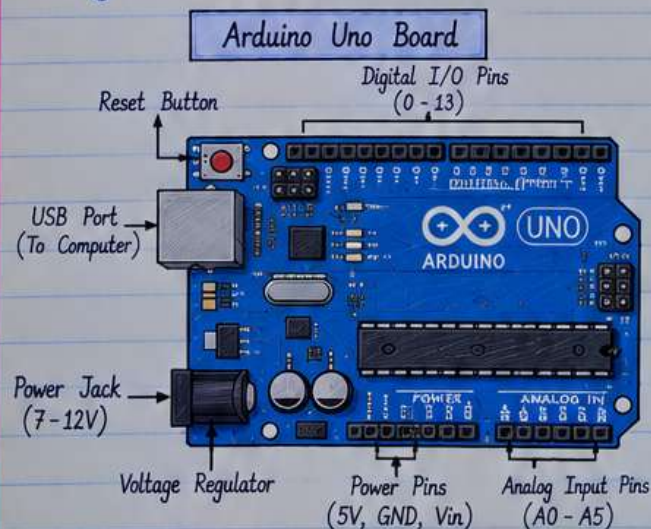
- Arduino Boards की कीमत बहुत कम होती है जिससे यह Students और Hobbyists के लिए सुलभ है।
- इसकी Low cost के बावजूद इसमें Powerful Microcontroller और कई Features मिलते हैं।
- विभिन्न प्रकार के Projects (Small to Advanced) के लिए यह एक Budget-friendly विकल्प है।

## 5. Multiple Input/Output Pins :

- Arduino में Digital और Analog दोनों प्रकार के Input/Output Pins उपलब्ध होते हैं।
- इन Pins के माध्यम से Sensors (जैसे Temperature, Light, Motion) को Read किया जा सकता है और Actuators (LED, Motor, Relay) को Control किया जा सकता है।
- कुछ Boards (जैसे Arduino Uno) में 14 Digital Pins और 6 Analog Pins होते हैं।

## 6. Built-in Communication Interfaces :

- Arduino में कई Communication Protocols सपोर्ट करते हैं जैसे UART (Serial), I2C, SPI आदि।
- इन Protocols के द्वारा विभिन्न Devices (Sensors, Modules, Displays) से आसानी से Communication की जा सकती है।
- यह IoT Projects और Embedded Systems के लिए बहुत उपयोगी होता है।



## 7. Cross-Platform IDE :

- Arduino IDE Windows, Linux और macOS सभी Operating Systems पर काम करता है।

## 2. Easy to Use :

- Arduino को खासकर Beginners के लिए बनाया गया है।
- इसका IDE (Integrated Development Environment) बहुत Simple और User-friendly होता है।
- Code लिखना, Compile करना और Upload करना आसान होता है।
- Complex Hardware Knowledge की आवश्यकता नहीं होती है।
- Plug and Play के तरह काम करता है।

## 4. Wide Range of Boards :

- Arduino में विभिन्न प्रकार के Boards उपलब्ध हैं जैसे Arduino Uno, Nano, Mega, Mini, Leonardo आदि।
- हर Board का अपना Specific Use होता है।
- Users अपनी Requirement के अनुसार सही Board का चयन कर सकते हैं।
- इससे छोटे से लेकर बड़े और Complex Projects तक बनाए जा सकते हैं।

## 8. Real-time Prototyping :

- Arduino का उपयोग करके Ideas को जल्दी से Prototype में बदला जा सकता है।
- Hardware और Software को एक साथ Test करना आसान होता है।
- इसका उपयोग Education, Research और Industry में Prototyping के लिए व्यापक रूप से किया जाता है।

## 9. Large Community Support :

- Arduino की एक बहुत बड़ी Global Community है।
- Online Forums, Tutorials, Libraries और Example Projects आसानी से उपलब्ध हैं।
- किसी भी समस्या के लिए Help और Guidance जल्दी प्राप्त हो जाती है।

## 10. Versatility and Applications :

- Arduino का उपयोग अनेक क्षेत्रों में किया जाता है जैसे Home Automation, Robotics, IoT, Industrial Control, Smart Agriculture, Weather Monitoring आदि।
- यह छोटे Simple Projects से लेकर Advanced Embedded Systems तक में उपयोग किया जा सकता है।
- इसकी Flexibility के कारण यह एक Versatile Development Platform है।

### Common Arduino Boards :

| Board Name       | Key Feature / Use                    |
|------------------|--------------------------------------|
| Arduino Uno      | Most popular, suitable for beginners |
| Arduino Nano     | Small size, compact projects         |
| Arduino Mega     | More I/O pins, advanced projects     |
| Arduino Mini     | Very small, embedded applications    |
| Arduino Leonardo | Built-in USB, used in custom devices |

# Topic : Components of Arduino Board

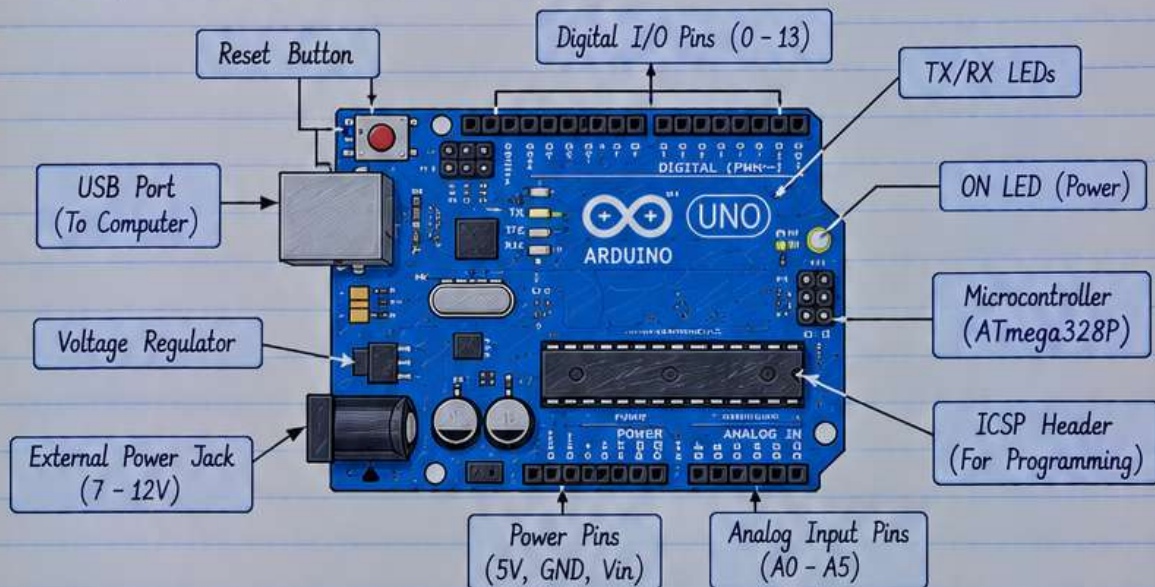
## 1. Introduction :

- Arduino Board एक Microcontroller based Development Board हैं जिसमें विभिन्न Hardware Components जुड़े होते हैं।
- ये Components मिलकर Arduino को Program करने, Input लेने, Output देने और बाहरी Devices के साथ Communication करने में मदद करते हैं।
- हर Component का अपना specific कार्य होता है और सभी मिलकर Board को एक complete development platform बनाते हैं।

### Arduino Board (Uno)



## 2. Components of Arduino Uno Board :



## 3. Description of Each Component :

### ① Microcontroller (ATmega328P):

- Arduino Uno में ATmega328P Microcontroller होता है।
- यह Program को Store करता है और Execute करता है।
- यह सभी Input/Output को Control करता है।

### ② USB Port :

- Computer से Connect करने के लिए USB Port का उपयोग होता है।
- इसके द्वारा Code Upload किया जाता है तथा Serial Communication भी होती है।
- यह 5V Power भी प्रदान करता है।

### ③ External Power Jack :

- यदि USB से Power उपलब्ध नहीं है तो External 7-12V DC Supply इस Jack से दी जा सकती है।
- यह Power को Voltage Regulator के माध्यम से 5V में बदलता है।

### ④ Voltage Regulator :

- यह External Supply (7-12V) को Regulate करके Board के लिए Stable 5V प्रदान करता है।
- यह सुनिश्चित करता है कि सभी Components को सही Voltage मिले।

### ⑤ Digital I/O Pins (0 - 13) :

- ये 14 Digital Pins होते हैं।
- इनका उपयोग Digital Input (जैसे Switch) और Digital Output (जैसे LED) के लिए किया जाता है।
- कुछ Pins PWM Output भी Support करते हैं (~ चिह्न के साथ)।

### ⑥ Analog Input Pins (A0 - A5) :

- ये 6 Analog Input Pins होते हैं।
- इनका उपयोग Analog Sensors (जैसे Temperature, Light, Sound) से Input लेने के लिए किया जाता है।
- ये 10-bit ADC के माध्यम से Analog Signal को Digital Value में Convert करते हैं।

### ⑦ Power Pins :

- इसमें 5V, GND और Vin Pins होते हैं।
- 5V : Regulated 5V Output (External Devices के लिए)
- GND : Ground Connection (सभी Circuits के लिए)
- Vin : External Supply Input (7-12V)

### ⑧ Reset Button :

- इस Button को दबाने से Microcontroller Restart हो जाता है।
- नया Program Upload करने के लिए उपयोगी है।

### ⑨ TX/RX LEDs :

- ये LEDs Serial Communication की स्थिति बताती हैं।
- TX LED : Data Send होने पर Blink करती है।
- RX LED : Data Receive होने पर Blink करती है।

### ⑩ ON LED :

- यह LED बताती है कि Board को Power मिल रही है।
- जब Board ON होता है तो यह LED जलती है।

### ⑪ ICSP Header :

- यह In-Circuit Serial Programming के लिए उपयोग होता है।
- इसके द्वारा Bootloader को Direct Program किया जा सकता है।

# Topic : Understanding of basic of Arduino IDE

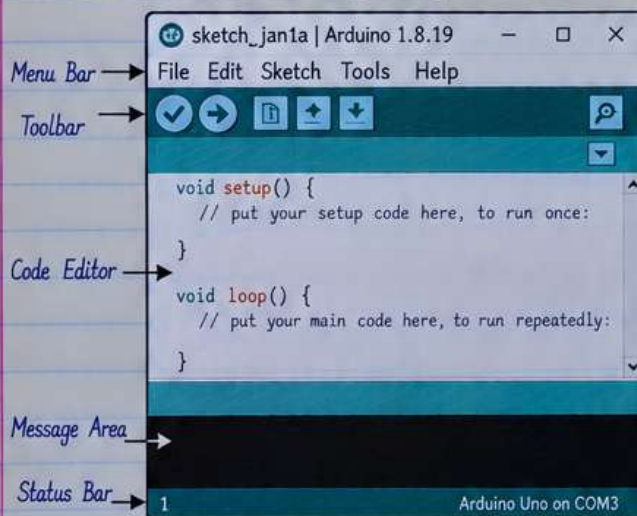
## 1. What is Arduino IDE ?

- **Arduino IDE (Integrated Development Environment)** एक Open Source Software हैं जिसे Arduino Boards के लिए Program लिखने, Compile करने और Upload करने के लिए उपयोग किया जाता है।
- यह C और C++ पर आधारित हैं और इसमें Arduino के लिए कुछ Special Libraries उपलब्ध होती हैं।
- Arduino IDE का Interface सरल और User-friendly होता है, इसलिए Beginners के लिए भी इसे सीखना आसान है।
- इसके द्वारा हम Sensors, Motors, LEDs और अन्य Devices को Control करने के लिए Code लिखते हैं और उसे Arduino Board में Upload करते हैं।

## 2. Features of Arduino IDE :

- Simple और Easy to Use Interface
- Syntax Highlighting (Code को अलग-अलग रंगों में दिखाता है)
- Auto Indentation
- Built-in Examples
- Library Manager (नई Libraries को Install करने के लिए)
- Multiple Board Support (Uno, Nano, Mega, etc.)
- Serial Monitor (Output देखने के लिए)
- Error Messages (Code में गलती होने पर)
- Cross-platform (Windows, macOS, Linux)

## 3. Arduino IDE Interface :



## 4. Main Parts and Their Functions :

| Part         | Function                                                      |
|--------------|---------------------------------------------------------------|
| Menu Bar     | File, Edit, Sketch, Tools, Help जैसे Options उपलब्ध होते हैं। |
| Toolbar      | Compile, Upload, New, Open, Save जैसे Shortcuts होते हैं।     |
| Code Editor  | यहाँ पर हम अपना Program (Sketch) लिखते हैं।                   |
| Message Area | Compile और Upload के समय Messages यहाँ पर दिखते हैं।          |
| Status Bar   | Connected Board और Port की जानकारी दिखाता है।                 |

## 5. Common Toolbar Icons :

|  |                  |                                                    |
|--|------------------|----------------------------------------------------|
|  | Verify / Compile | Code को Compile करता है और Error Check करता है।    |
|  | Upload           | Compiled Code को Arduino Board में Upload करता है। |
|  | New              | नया Sketch बनाता है।                               |
|  | Open             | पहले से बने Sketch को Open करता है।                |
|  | Save             | Current Sketch को Save करता है।                    |
|  | Serial Monitor   | Board से मिलने वाले Output को देखने के लिए।        |

## 6. Arduino Sketch Structure :

|                                                                                              |                                    |
|----------------------------------------------------------------------------------------------|------------------------------------|
| <pre>void setup() {   // यह भाग एक बार चलता है   // यहाँ हम Initialization करते हैं }</pre>  | Setup Function<br>(Runs Once)      |
| <pre>void loop() {   // यह भाग बार-बार चलता है   // यहाँ हम अपना Main Code लिखते हैं }</pre> | Loop Function<br>(Runs Repeatedly) |

## 7. Steps to Run a Program in Arduino IDE :

- ① Arduino IDE को Open करें।
- ② सही Board (Tools > Board) को Select करें।
- ③ सही Port (Tools > Port) को Select करें।
- ④ अपना Code लिखें या Example Open करें।
- ⑤ Verify (✓) पर Click करके Compile करें।
- ⑥ Upload (→) पर Click करके Code को Board में भेजें।

## 8. Useful Menu Options :

| Menu Option | Use / Function                                    |
|-------------|---------------------------------------------------|
| File        | New, Open, Save, Examples, Exit                   |
| Edit        | Copy, Paste, Find, Select All                     |
| Sketch      | Verify/Compile, Upload, Include Library, Add File |
| Tools       | Board, Port, Programmer, Burn Bootloader          |
| Help        | Getting Started, Reference, Examples              |

# Topic : Arduino Programming Language (C Language)

## 1. Introduction :

- Arduino की Programming C और C++ पर आधारित होती है।
- Arduino IDE में हम C/C++ जैसे ही Simple C Language का उपयोग करते हैं।
- यह भाषा Embedded Systems के लिए Optimized होती है।
- Arduino में लिखे गए Program को Sketch कहा जाता है।
- C Language का उपयोग इसलिए किया जाता है क्योंकि यह Fast, Reliable और Hardware Level पर काम करने के लिए उपयुक्त है।

## 2. Structure of Arduino Program (Sketch):

```
void setup() {  
  // यह भाग केवल एक बार चलता है  
  // यहाँ Initialization करते हैं  
}
```

Setup Function  
(Runs Once)

```
void loop() {  
  // यह भाग बार-बार चलता है  
  // यहाँ Main Code (Repeatedly)  
  लिखते हैं।  
}
```

Loop Function  
(Runs Repeatedly)

## 3. Basic Syntax :

- Program की हर line का अंत ; (Semicolon) से होता है।
- { } को Block लिखने के लिए उपयोग किया जाता है।
- Comments लिखने के लिए // (Single line) और /\* ... \*/ (Multi-line) का उपयोग होता है।
- C Language Case Sensitive होती है (जैसे LED और led अलग माने जाते हैं)।

## 4. Data Types (Variables):

| Data Type | Size (Approx.) | Use / Example               |
|-----------|----------------|-----------------------------|
| int       | 2 bytes        | Integer value (जैसे 10, -5) |
| float     | 4 bytes        | Decimal value (जैसे 3.14)   |
| char      | 1 byte         | Single character (जैसे 'A') |
| boolean   | 1 byte         | True / False (HIGH / LOW)   |
| String    | Varies         | Text data (जैसे "Hello")    |

## 5. Constants :

- HIGH → 5V (ON)
- LOW → 0V (OFF)
- INPUT → Pin को Input के रूप में सेट करने के लिए
- OUTPUT → Pin को Output के रूप में सेट करने के लिए
- Serial → Serial Communication के लिए उपयोग होता है।

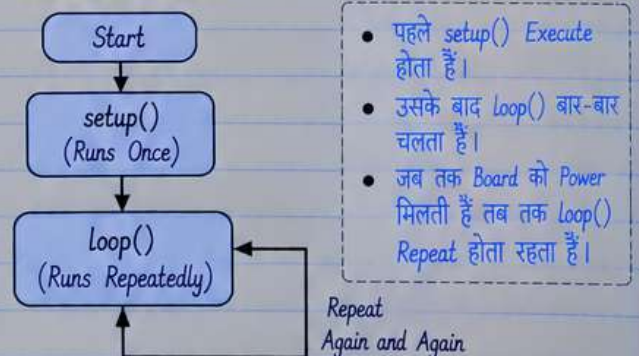
## 6. Important Functions :

| Function         | Use / Description                         | Example                      |
|------------------|-------------------------------------------|------------------------------|
| pinMode()        | Pin की mode set करता है                   | pinMode(13, OUTPUT);         |
| digitalWrite()   | Pin को HIGH या LOW करता है                | digitalWrite(13, HIGH);      |
| digitalRead()    | Pin से Input पढ़ता है                     | int val = digitalRead(2);    |
| analogWrite()    | PWM Output देता है (0-255)                | analogWrite(9, 120);         |
| analogRead()     | Analog Input पढ़ता है (0-1023)            | int sensor = analogRead(A0); |
| delay()          | कुछ समय के लिए रुकता है (millisecond में) | delay(1000);                 |
| Serial.begin()   | Serial Communication शुरू करता है         | Serial.begin(9600);          |
| Serial.print()   | Serial Monitor पर Data भेजता है           | Serial.print("Hello");       |
| Serial.println() | नई Line के साथ Data भेजता है              | Serial.println(val);         |

## 7. Example Program (LED Blink):

```
// LED Blink Example  
void setup() {  
  pinMode(13, OUTPUT); // Pin 13 को Output बनाना  
}  
void loop() {  
  digitalWrite(13, HIGH); // LED ON  
  delay(1000); // 1 Second रुकना  
  digitalWrite(13, LOW); // LED OFF  
  delay(1000); // 1 Second रुकना  
}
```

## 8. Program Flow :



## 9. Comments :

- Single Line Comment : // यह एक Comment है
- Multi-line Comment : /\* यह कई Line का Comment है \*/

# Topic : Variables

## 1. What is a Variable ?

- Variable एक ऐसा नाम (name) होता है जो Memory में Data को store करने के लिए उपयोग किया जाता है।
- इसमें Data का value समय-समय पर बदल सकता है।
- यह Program में Input, Processing और Output के बीच डेटा को store और manipulate करने में मदद करता है।
- हर Variable का एक Data Type होता है जो यह बताता है कि इसमें किस प्रकार का Data store होगा।

## 2. Why Variables are Used ?

- Data को store करने के लिए।
- Program को flexible बनाने के लिए।
- Input से प्राप्त data को temporarily रखने के लिए।
- Processing के दौरान values को change करने के लिए।
- Code को readable और maintainable बनाने के लिए।
- Real-world applications (Sensors, User Input, etc.) में data को handle करने के लिए।

## 3. Syntax (Declaration & Initialization) :

```
dataType variableName;           // Declaration
dataType variableName = value;    // Initialization
```

### Example :

```
int age;                          // Declaration
int age = 20;                      // Initialization
float temperature = 36.5;
char grade = 'A';
boolean isOn = true;
String name = "Brajant";
```

## 4. Rules for Naming Variables :

- Variable name letters (A-Z, a-z), digits (0-9) और underscore (\_) से मिलकर बन सकता है।
- Variable name की शुरुआत letter या underscore से करनी चाहिए (digit से नहीं)।
- Variable name case sensitive होता है (age और Age दोनों अलग-अलग हैं)।
- Space या special characters (@, #, \$, etc.) का उपयोग नहीं किया जा सकता।
- Variable name कोई reserved keyword (int, float, etc.) नहीं होना चाहिए।
- अर्थपूर्ण नाम (meaningful name) का उपयोग करना अच्छी practice होती है।

## 5. Data Types in Arduino (C Language) :

| Data Type    | Size (Approx.) | Range / Description                        | Example                               |
|--------------|----------------|--------------------------------------------|---------------------------------------|
| int          | 2 bytes        | -32,768 to 32,767                          | <code>int count = 10;</code>          |
| unsigned int | 2 bytes        | 0 to 65,535                                | <code>unsigned int s = 100;</code>    |
| long         | 4 bytes        | -2,147,483,648 to 2,147,483,647            | <code>long distance = 1000;</code>    |
| float        | 4 bytes        | Decimal numbers (6-7 digits)               | <code>float temp = 25.5;</code>       |
| double       | 4 bytes        | Decimal numbers (same as float in Arduino) | <code>double val = 3.1416;</code>     |
| char         | 1 byte         | Single character                           | <code>char grade = 'A';</code>        |
| boolean      | 1 byte         | true or false                              | <code>boolean flag = true;</code>     |
| String       | Varies         | Text data (String)                         | <code>String name = "Arduino";</code> |

## 6. Example Program :

```
void setup() {
  int count = 5;
  float voltage = 3.7;
  char letter = 'B';
  boolean ledStatus = true;
  String message = "Hello Arduino";
  Serial.begin(9600);
  Serial.print("Count: ");
  Serial.println(count);
  Serial.print("Voltage: ");
  Serial.println(voltage);
  Serial.print("Letter: ");
  Serial.println(letter);
  Serial.print("LED Status: ");
  Serial.println(ledStatus);
  Serial.print("Message: ");
  Serial.println(message);
}
```

## 7. Changing the Value of a Variable :

- Program के दौरान variable का value बदला जा सकता है।
- Same variable में नया value assign किया जा सकता है।
- यह Loop, Condition या User Input के अनुसार बदलता है।

### Example :

```
int count = 0;
count = 10;           // नया value assign
count = count + 5;    // value update
```

## 8. Types of Variables (Usage के आधार पर) :

- Global Variable : जो सभी functions में accessible होता है।
- Local Variable : जो केवल एक specific function में उपयोग होता है।
- Static Variable : जो अपना value बनाए रखता है, भले ही function बार-बार call हो।

## Topic : Data Types (in Arduino / C Language)

### 1. What are Data Types ?

- Data Types उस प्रकार के बारे में बताते हैं कि किसी Variable में किस प्रकार का Data store होगा।
- हर Variable का एक निश्चित Data Type होता है।
- Data Type यह निर्धारित करता है कि Variable कितनी Memory लेगा और उसमें किस प्रकार का value store किया जा सकता है।
- Arduino में C/C++ के Data Types का ही उपयोग होता है।
- सही Data Type चुनना जरूरी होता है क्योंकि इससे Memory का सही उपयोग और Program की Better Performance मिलती है।

### 4. Common Data Types in Arduino :

| Data Type     | Size (Approx.) | Range / Description                        | Example                   |
|---------------|----------------|--------------------------------------------|---------------------------|
| int           | 2 bytes        | -32,768 to 32,767                          | int x = 100;              |
| unsigned int  | 2 bytes        | 0 to 65,535                                | unsigned int x = 500;     |
| long          | 4 bytes        | -2,147,483,648 to 2,147,483,647            | long x = 100000;          |
| unsigned long | 4 bytes        | 0 to 4,294,967,295                         | unsigned long x = 100000; |
| float         | 4 bytes        | Decimal numbers (6-7 digits)               | float x = 3.14;           |
| double        | 4 bytes        | Decimal numbers (same as float in Arduino) | double x = 3.1416;        |
| char          | 1 byte         | Single character                           | char ch = 'A';            |
| boolean       | 1 byte         | true or false                              | boolean flag = true;      |
| String        | Varies         | Text data (String)                         | String name = "Arduino";  |

### 2. Categories of Data Types :

- ① Primary / Built-in Data Types
  - int, float, char, boolean, etc.
- ② Derived Data Types
  - array, pointer, etc.
- ③ User-defined Data Types
  - structure, union, enum, etc.

### 3. Key Points :

- हर Data Type का size अलग होता है।
- यह signed (positive और negative) या unsigned (केवल positive) हो सकता है।

- कुछ Data Types decimal value store करते हैं (जैसे float), जबकि कुछ integer value store करते हैं (जैसे int)।
- Character data के लिए char और true/false के लिए boolean उपयोग होता है।

### 5. Examples :

```
int age = 20;           // Integer value
unsigned int count = 50; // Only positive value
long distance = 100000;  // Large integer value
float temperature = 36.5; // Decimal value
double pi = 3.1416;      // More precision
char grade = 'B';        // Single character
boolean isOn = false;    // true or false
String msg = "Hello";    // Text data
```

### 6. Memory Usage Illustration :

1 byte (char / boolean) 

2 bytes (int / unsigned int) 

4 bytes (long / float / double) 

String (Varies) 

### 7. Type Conversion :

- एक Data Type से दूसरे Data Type में value बदलने की प्रक्रिया को Type Conversion कहते हैं।
- यह Automatic (Compiler द्वारा) या Manual (Type Casting द्वारा) हो सकता है।

```
int x = 10;
float y = x;           // Automatic conversion
int z = (int)3.14;     // Manual type casting
```

# Topic : Loops

## 1. What is a Loop ?

- Loop एक control structure है, जिसके द्वारा हम किसी code block को बार-बार execute कर सकते हैं।
- यह तब उपयोगी होता है जब हमें किसी काम को multiple times करना होता है, जैसे 1 से 10 तक numbers print करना।
- Loop से code छोटा, readable और efficient बनता है क्योंकि हमें बार-बार same code लिखने की जरूरत नहीं होती है।

## 2. Types of Loops :

- ① for loop - जब हमें निश्चित number of times loop चलाना हो।
- ② while loop - जब तक condition true है, तब तक loop चलता है।
- ③ do...while loop - पहले code execute होता है, फिर condition check होती है।

## 3. For Loop :

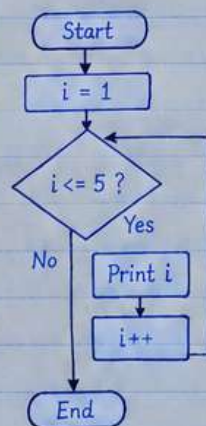
- for loop का उपयोग तब करते हैं जब हमें exact number of times loop चलाना हो।
- इसमें initialization, condition और increment/decrement तीनों एक ही line में लिखे जाते हैं।

### Syntax :

```
for (initialization; condition; increment) {  
    // code  
}
```

Example : 1 to 5 तक print करना

```
for (int i = 1; i <= 5; i++) {  
    printf("%d ", i);  
}
```



## 4. While Loop :

- while loop तब तक चलता है जब तक condition true रहती है।
- यह loop तब उपयोगी है जब हमें पहले से number of iterations पता न हो।

### Syntax :

```
while (condition) {  
    // code  
}
```

Example : 1 से 5 तक print करना

```
int i = 1;  
while (i <= 5) {  
    printf("%d ", i);  
    i++;  
}
```

## 5. Do...While Loop :

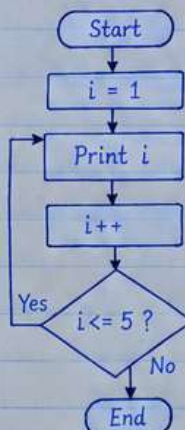
- इस loop में पहले code execute होता है और फिर condition check होती है।
- यह loop कमसे एक बार जरूर चलता है, चाहे condition false ही क्यों न हो।

### Syntax :

```
do {  
    // code  
} while (condition);
```

Example : 1 से 5 तक print करना

```
int i = 1;  
do {  
    printf("%d ", i);  
    i++;  
} while (i <= 5);
```



## 6. Loop Control Statements :

- Break - loop को बीच में ही terminate कर देता है।
- Continue - current iteration को skip करके अगली iteration पार जाता है।

### Example :

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue; // 3 को skip करेगा  
    }  
    printf("%d ", i);  
}
```

## 7. Nested Loops :

- एक loop के अंदर दूसरा loop लिखने nested loop कहलाता है।
- यह तब उपयोगी होता है जब हमें 2D data या table pattern print करना हो।
- जैसे - 3x3 star pattern

### Example :

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        printf("*");  
    }  
    printf("\n");  
}
```

### Output :

```
* * *  
* * *  
* * *
```

## 8. Loop with Arrays (Quick Example) :

- Array के सभी elements को print करने के लिए loop का उपयोग करते हैं।

### Example :

```
int arr[5] = {10, 20, 30, 40, 50};  
for (int i = 0; i < 5; i++) {  
    printf("%d ", arr[i]);  
}
```

# Topic : Control Statement

## 1. Introduction :

- Control Statement का उपयोग Program के flow को control करने के लिए किया जाता है।
- यह decide करता है कि किस condition में कौन सा block of code execute होगा।
- Control Statement की मदद से Program को decision making, repetition और jump करने की capability मिलती है।
- इसके द्वारा Program का flow logical और organized बनता है।

## 2. Types of Control Statements :

- Selection / Decision Making Statements  
(if, if-else, else-if, switch)
- Iteration / Looping Statements  
(for, while, do-while)
- Jump Statements  
(break, continue, return, goto)

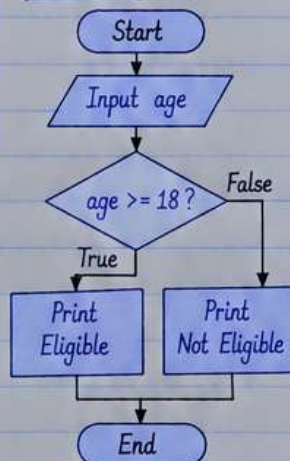
## 3. Selection Statements (if-else) :

- Selection statement condition के आधार पर true या false होने पर अलग-अलग block execute करता है।
- यह Program में decision making के लिए उपयोग होता है।

### Example :

```
int age = 18;
if (age >= 18) {
    printf("You are eligible to vote");
} else {
    printf("You are not eligible");
}
```

### Flowchart :



## 4. switch Statement :

- switch statement का उपयोग तब किया जाता है जब एक variable के multiple values को check करना हो।
- यह if-else की तुलना में ज्यादा readable और easy होता है।

### Example :

```
int day = 2;
switch(day) {
    case 1: printf("Monday"); break;
    case 2: printf("Tuesday"); break;
    case 3: printf("Wednesday"); break;
    default: printf("Invalid day");
}
```

## 5. Iteration / Looping Statements :

- Loop statements का उपयोग किसी block of code को बार-बार execute करने के लिए किया जाता है।
- यह repetitive tasks को perform करने में मदद करता है।

| for loop                                                             | while loop                                                                 | do-while loop                                                                  |
|----------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <pre>for (int i = 1; i &lt;= 5; i++) {     printf("%d ", i); }</pre> | <pre>int i = 1; while (i &lt;= 5) {     printf("%d ", i);     i++; }</pre> | <pre>int i = 1; do {     printf("%d ", i);     i++; } while (i &lt;= 5);</pre> |

## 6. Jump Statements :

- Jump statement का उपयोग loop या function के flow को change करने के लिए किया जाता है।
- यह Program के execution को एक specific point पर transfer कर देता है।

| Statement | Use                                             |
|-----------|-------------------------------------------------|
| break     | loop या switch को terminate करता है।            |
| continue  | current iteration को skip करता है।              |
| return    | function से value return करता है।               |
| goto      | program के किसी specific label पर jump करता है। |

## 7. Control Statements का महत्व :

- Program के flow को control और manage करने में मदद करते हैं।
- Large programs को logical और structured बनाते हैं।
- Decision making और repetitive tasks को easily implement करने की सुविधा देते हैं।
- यह Program की efficiency और readability को बढ़ाते हैं।

# Topic : Function

## 1. What is a Function ?

- Function एक group of statements का नाम है, जो किसी specific task को perform करने के लिए लिखा जाता है।
- Function का use code को modular बनाने, repeat करने और manage करने के लिए होता है।
- एक बार function लिखने के बाद उसे program में कहीं भी call करके इस्तेमाल किया जा सकता है।
- Function Program की readability को बढ़ाता है और divide and conquer approach को follow करता है।

## 2. Advantages of Function :

- Code reusability बढ़ता है।
- Program का size कम होता है।
- Code को modular बनाता है।
- Debugging और testing आसान होती है।
- Large programs को manage करना आसान होता है।
- Readability और maintainability बेहतर होती है।

## 3. Types of Functions :

- ① User Defined Function (हम स्वयं बनाते हैं)
- ② Library Function (पहले से उपलब्ध)

## 4. Syntax of Function :

```
returnType functionName(parameter list) {  
    // function body (statements)  
    return value; // (optional)  
}
```

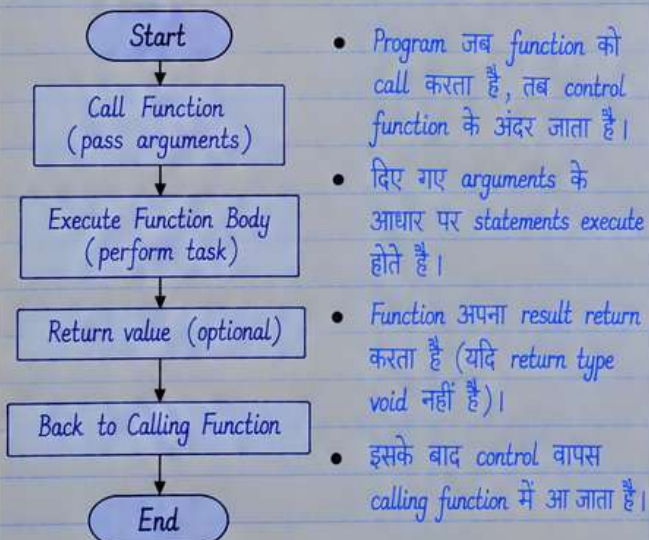
- returnType : function द्वारा return किए जाने वाले data type को define करता है।
- functionName : यह function का नाम होता है।
- parameter list : function को दिए जाने वाले input values (arguments) को दर्शाता है।
- return value : function अपना result वापस करता है।

## 5. Example (User Defined Function) :

```
int add(int a, int b) {  
    int sum = a + b;  
    return sum;  
}  
  
void setup() {  
    Serial.begin(9600);  
    int result = add(10, 20);  
    Serial.println(result);  
}  
  
void loop() {  
}
```

यह function दो numbers का sum return करता है।

## 6. Working of Function (Flow Diagram) :



## 7. Types of Function (with Example) :

| Type                         | Description                              | Example                                                        |
|------------------------------|------------------------------------------|----------------------------------------------------------------|
| No Argument<br>No Return     | कोई input नहीं,<br>कोई value return नहीं | <pre>void show() {<br/>    Serial.println("Hi");<br/>}</pre>   |
| With Argument<br>No Return   | input मिलता है,<br>कोई value return नहीं | <pre>void show(int x) {<br/>    Serial.println(x);<br/>}</pre> |
| No Argument<br>With Return   | कोई input नहीं,<br>value return करता है  | <pre>int getValue() {<br/>    return 10;<br/>}</pre>           |
| With Argument<br>With Return | input मिलता है,<br>value return करता है  | <pre>int square(int n) {<br/>    return n * n;<br/>}</pre>     |

## 8. Library Function :

- ये pre-defined functions होते हैं, जो Arduino/C library में पहले से मौजूद रहते हैं।
- इनका use specific tasks को आसानी से करने के लिए किया जाता है।
- उदाहरण : Serial.begin(), Serial.println(), delay(), pinMode(), digitalWrite() आदि।

## 9. Important Points :

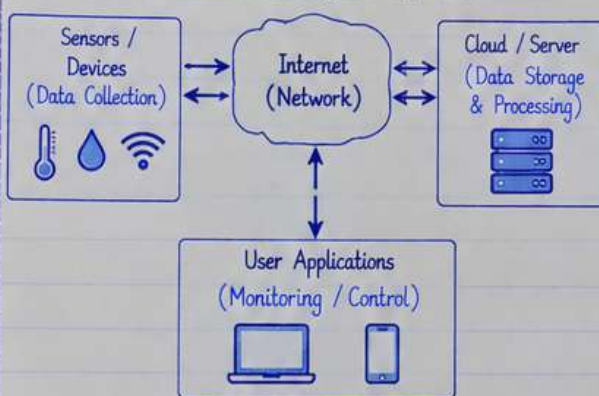
- Function का नाम meaningful होना चाहिए।
- Parameter का data type सही होना चाहिए।
- Return type और return value का type match होना चाहिए।
- Function को call करने के लिए उसका नाम और arguments सही तरीके से लिखने चाहिए।

# Unit - 5 : IoT and M2M

## 1. Introduction to IoT (Internet of Things)

- Internet of Things (IoT) एक ऐसा concept है जिसमें विभिन्न physical devices, sensors, software और other objects को internet के माध्यम से connect किया जाता है ताकि वे data collect, exchange और analyze कर सकें।
- IoT का मुख्य उद्देश्य everyday devices को smart बनाना और उन्हें internet के माध्यम से communicate करने में सक्षम करना है।
- IoT के द्वारा हम real-time data प्राप्त कर सकते हैं, remote monitoring कर सकते हैं और processes को automate कर सकते हैं।

### IoT Working Diagram



### 1.1 Characteristics of IoT

- Connectivity - devices are connected via internet.
- Sensing - data is collected using sensors.
- Intelligence - data is processed to get useful information.
- Automation - tasks are automatically performed.
- Scalability - easily expandable to large number of devices.
- Real-time Monitoring - live data access and control.

### 1.2 Components of IoT

- Sensors - temperature, humidity, motion, etc. को sense करते हैं।
- Devices - data collect और transmit करते हैं।
- Connectivity - Wi-Fi, Bluetooth, Zigbee, Cellular आदि के द्वारा connection होता है।
- Cloud / Server - data को store और process करता है।
- Application - user को data visualize करने और control के लिए interface प्रदान करता है।

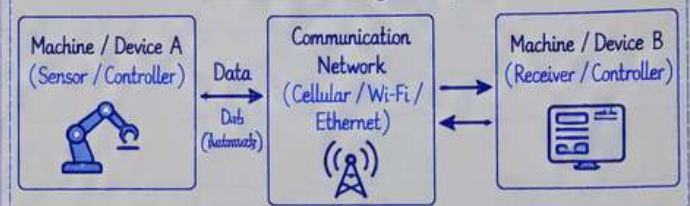
### 1.3 Applications of IoT

- Smart Home - home appliances का remote control
- Smart City - traffic, street lights, waste management
- Healthcare - patient monitoring devices
- Agriculture - soil moisture, weather monitoring
- Industrial IoT (IIoT) - machine monitoring, predictive maintenance
- Smart Wearables - fitness bands, smart watches

## 2. Introduction to M2M (Machine to Machine)

- Machine to Machine (M2M) एक ऐसी technology है जिसमें दो या दो से अधिक machines / devices बिना मानव हस्तक्षेप के आपस में communicate करते हैं और data exchange करते हैं।
- M2M का प्रमुख उद्देश्य devices के बीच automatic communication और data transfer को संभव बनाना है। ताकि monitoring, control और management अधिक efficient हो सके।
- M2M, IoT का एक महत्वपूर्ण हिस्सा है, लेकिन M2M आमतौर पर specific और closed system में काम करता है जबकि IoT का scope अधिक व्यापक है।

### M2M Working Diagram



### 2.1 Key Features of M2M

- Automatic Communication - बिना मानव हस्तक्षेप के devices के बीच data exchange.
- Wired / Wireless Connectivity - Cellular, Wi-Fi, Ethernet, Bluetooth आदि का उपयोग।
- Remote Monitoring and Control - दूर से monitoring और control संभव।
- Increased Efficiency - manual intervention की आवश्यकता कम होती है।
- Real-time Data Transfer - live data का आदान-प्रदान।
- Secure Communication - data security महत्वपूर्ण होती है।

### 2.2 Difference between IoT and M2M

| Feature            | IoT                                             | M2M                                                   |
|--------------------|-------------------------------------------------|-------------------------------------------------------|
| Meaning            | Network of smart devices connected via internet | Direct communication between machines                 |
| Scope              | Wide (consumers + industry + many devices)      | Limited (specific applications)                       |
| Human Intervention | Can be with or without human intervention       | Generally without human intervention                  |
| Architecture       | Often uses cloud and big data analytics         | Usually point-to-point or closed system               |
| Data Usage         | Advanced analytics, AI, decision making         | Basic data transfer and control                       |
| Examples           | Smart home, smart city, wearables, healthcare   | ATM monitoring, industrial machines, vending machines |

### 2.3 Applications of M2M

- Industrial Automation - machines की monitoring और control
- Smart Metering - electricity, water, gas meters
- Telematics - vehicle tracking और fleet management
- Healthcare Devices - remote patient monitoring
- Vending Machines - inventory और payment status की जानकारी
- Security Systems - surveillance और alarm systems

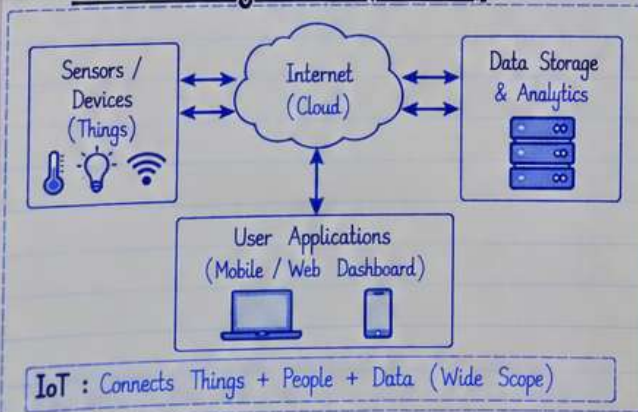
# Difference between IoT and M2M

IoT (Internet of Things) और M2M (Machine to Machine) दोनों technologies devices के बीच communication और data exchange की सुविधा प्रदान करती हैं। लेकिन इन दोनों में कुछ महत्वपूर्ण अंतर हैं, जो उनके scope, architecture, applications और purpose के आधार पर समझे जा सकते हैं।

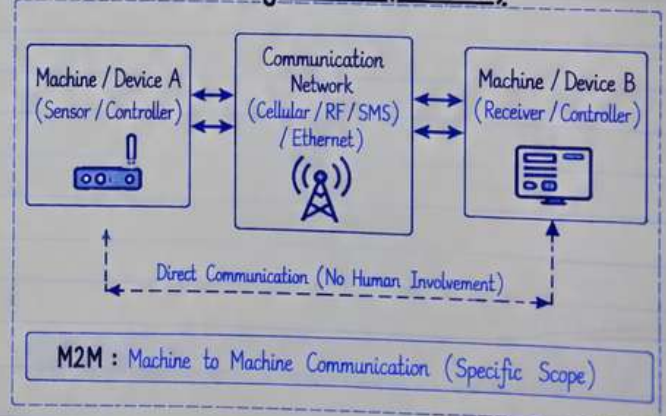
## 1. Comparison Table

| S. No. | Parameter                 | IoT (Internet of Things)                                                                                                       | M2M (Machine to Machine)                                                              |
|--------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| ①      | Full Form                 | Internet of Things                                                                                                             | Machine to Machine                                                                    |
| ②      | Meaning                   | Physical devices, sensors और objects का network जो internet के माध्यम से connect होकर data collect, share और analyze करते हैं। | दो machines / devices के बीच direct communication, बिना human intervention के।        |
| ③      | Scope                     | Wide scope (consumer, industrial, healthcare, smart home, smart city आदि)                                                      | Limited scope (specific and closed systems)                                           |
| ④      | Internet Dependency       | Mostly internet based (cloud, analytics, remote access)                                                                        | Can work with or without internet (often uses cellular, SMS, RF, etc.)                |
| ⑤      | Architecture              | Complex architecture with sensors, gateways, cloud, data analytics और applications                                             | Simple and direct communication between devices                                       |
| ⑥      | Communication             | Multiple protocols (Wi-Fi, Bluetooth, Zigbee, Cellular, Ethernet, etc.)                                                        | Often uses dedicated networks (Cellular, SMS, RF, etc.)                               |
| ⑦      | Data Handling             | Large amount of data, stored in cloud and analyzed using AI/ML                                                                 | Specific and limited data, usually predefined                                         |
| ⑧      | Human Involvement         | Can involve human interaction through mobile apps, dashboards, web portals                                                     | Usually no human intervention (required only for setup/maintenance)                   |
| ⑨      | Applications              | Smart home, wearable devices, smart cities, industrial IoT, healthcare, agriculture, smart vehicles etc.                       | Vending machines, ATM, utility metering, industrial automation, fleet management etc. |
| ⑩      | Flexibility & Scalability | Highly scalable and flexible                                                                                                   | Less scalable and limited to specific use cases                                       |
| ⑪      | Example                   | Smart thermostat, smart watch, smart agriculture system                                                                        | Automatic meter reading, vehicle tracking, industrial machine monitoring              |
| ⑫      | Goal                      | To create a smart and connected world by integrating devices, data and people                                                  | To enable direct and reliable communication between machines for a specific task      |

## 2. IoT Working Model (General)



## 3. M2M Working Model (General)



## 4. Conclusion

- IoT is a broad concept that includes M2M and many other technologies.
- M2M is a subset of IoT, focused only on machine-to-machine communication for specific applications.
- In short, All M2M systems are part of IoT, but all IoT systems are not M2M.

# SDN and NFV for IoT - Software Defined Networking

## 1. Introduction

- Software Defined Networking (SDN) एक modern networking approach है जिसमें network का control hardware devices की बजाय software द्वारा किया जाता है।
- यह network को flexible, programmable और centrally managed बनाता है।
- SDN का उपयोग IoT environments में विशेष रूप से उपयोगी है, जहाँ बड़ी संख्या में devices को efficiently manage करना होता है।

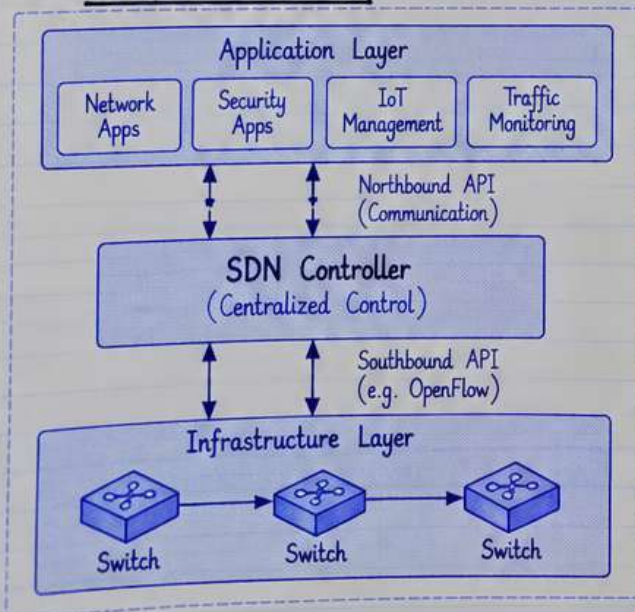
## 2. What is SDN ?

- SDN एक network architecture है जिसमें control plane और data plane को अलग किया जाता है।
- Network का control एक centralized software controller द्वारा किया जाता है।
- यह network devices (switches, routers) को simple forwarding devices की तरह work करने देता है।
- Policies, rules और network behavior software के through define किए जाते हैं।

## 3. Key Characteristics of SDN

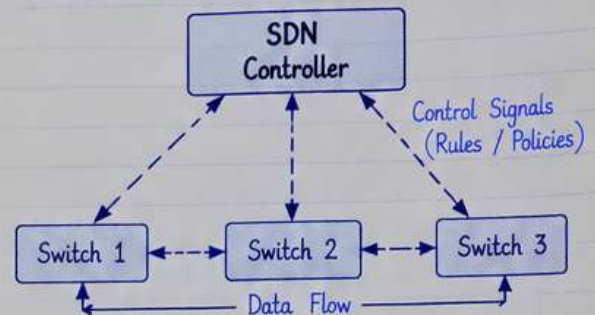
- ① Centralized Control - पूरा network एक controller द्वारा manage होता है।
- ② Separation of Control and Data Plane - control plane और data plane अलग-अलग होते हैं।
- ③ Programmability - network को software के द्वारा program किया जा सकता है।
- ④ Network Virtualization - logical networks create किए जा सकते हैं।
- ⑤ Dynamic Configuration - network changes को quickly apply किया जा सकता है।
- ⑥ Improved Management - monitoring और troubleshooting आसान हो जाता है।

## 4. SDN Architecture



## 5. How SDN Works ?

- ① Applications send network requirements to the controller (via Northbound API).
- ② Controller decides the best path and policy.
- ③ Controller sends instructions to network devices (via Southbound API, e.g. OpenFlow).
- ④ Devices forward the data as per the rules.



## 6. Benefits of SDN for IoT

- Better device management (large number of IoT devices)
- Improved network visibility and control
- Dynamic and scalable network
- Support for heterogeneous devices
- Enhanced security (centralized policy control)
- Cost-effective and efficient network operation

## 7. Role of SDN in IoT

- Manage huge number of IoT devices efficiently
- Enable real-time monitoring and traffic control
- Support smart applications (smart home, smart city, industry)
- Provide QoS (Quality of Service) for critical IoT data
- Help in network slicing for different IoT services
- Improve security by centralized control and policy enforcement

## 8. SDN vs Traditional Networking

| Feature           | Traditional Networking       | SDN                               |
|-------------------|------------------------------|-----------------------------------|
| Control           | Distributed (in each device) | Centralized (software controller) |
| Architecture      | Tightly coupled              | Decoupled control and data plane  |
| Configuration     | Manual (device-wise)         | Programmable (software)           |
| Flexibility       | Limited                      | High                              |
| Scalability       | Difficult                    | Easy                              |
| Device Management | Complex                      | Simplified                        |
| Cost              | Higher                       | Lower                             |
| Support for IoT   | Limited                      | Well suited                       |

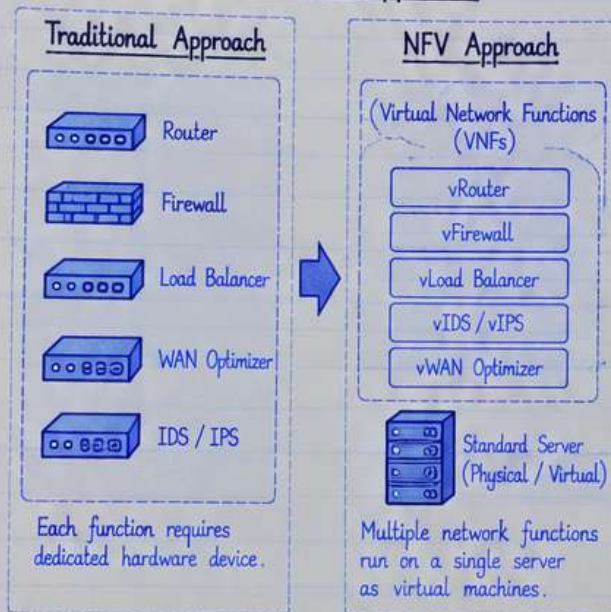
## 9. Conclusion

- SDN IoT networks के लिए एक powerful technology है जो network को flexible, scalable और easy to manage बनाता है।
- यह future IoT applications के लिए reliable और efficient communication infrastructure प्रदान करता है।

# Network Function Virtualization (NFV)

Network Function Virtualization (NFV) एक ऐसा concept है जिसमें traditional hardware based network functions को dedicated physical devices के बजाय virtualized software functions के रूप में implement किया जाता है। इसका उद्देश्य network को अधिक flexible, scalable और cost-effective बनाना है।

## 1. Traditional vs NFV Approach



## 2. What is NFV ?

- NFV (Network Function Virtualization) network functions जैसे firewall, router, load balancer, IDS/IPS आदि को software के रूप में virtual machines (VMs) या containers पर चलाने की तकनीक है।
- यह functions को general purpose hardware पर run करता है, जिससे अलग-अलग hardware devices की आवश्यकता नहीं होती है।
- NFV को ETSI (European Telecommunications Standards Institute) द्वारा standardize किया गया है।

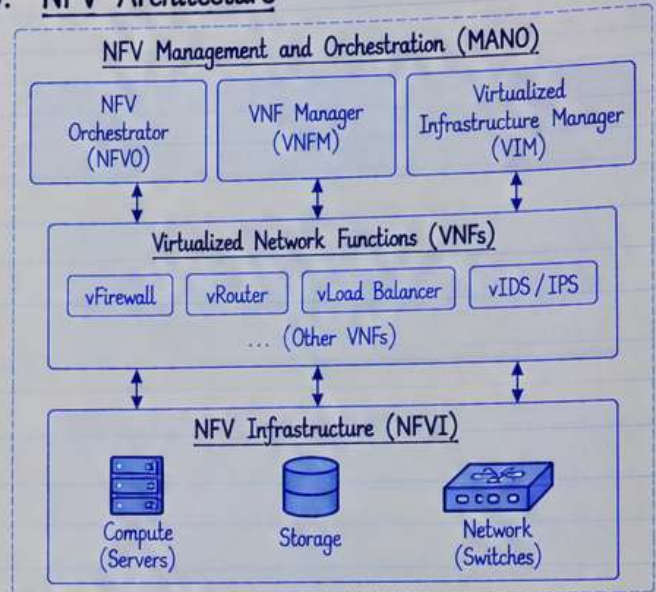
## 3. Key Characteristics of NFV

- ① Virtualization - network functions as software (VNFs)
- ② Runs on COTS Hardware - standard commercial off-the-shelf servers.
- ③ Flexibility - functions को आसानी से deploy, update और remove किया जा सकता है।
- ④ Scalability - demand के अनुसार resources को बढ़ाया या घटाया जा सकता है।
- ⑤ Automation - deployment और management को automate करता है।
- ⑥ Cost-effective - dedicated hardware की आवश्यकता कम होती है।
- ⑦ Improved Resource Utilization - एक ही hardware पर कई functions run हो सकते हैं।

## 4. Common Network Functions in NFV

- Virtual Firewall (vFW)
- Virtual Router (vRouter)
- Virtual Load Balancer (vLB)
- Virtual Private Network (vVPN)
- Intrusion Detection / Prevention System (vIDS / vIPS)
- Network Address Translation (vNAT)
- WAN Optimization
- Traffic Monitoring and Analytics

## 5. NFV Architecture



## 6. Benefits of NFV

- Lower Cost - dedicated hardware की आवश्यकता नहीं।
- Faster Deployment - services को जल्दी deploy किया जा सकता है।
- Scalability - demand के अनुसार resources को scale करना आसान।
- Flexibility - new services को easily add किया जा सकता है।
- Better Resource Utilization - एक ही hardware पर multiple functions run होते हैं।
- Simplified Management - centralized management और automation।
- Supports Innovation - नए network services को जल्दी develop और test किया जा सकता है।

## 7. Challenges of NFV

- Performance Overhead - virtualization के कारण performance कम हो सकता है।
- Security Issues - multi-tenant environment में security risk।
- Complex Management - orchestration और monitoring जटिल हो सकता है।
- Interoperability - different vendors के VNFs के बीच compatibility।
- Reliability - critical network functions के लिए high availability सुनिश्चित करना आवश्यक है।

## 8. NFV vs Traditional Networking

| Aspect               | Traditional Networking     | NFV                          |
|----------------------|----------------------------|------------------------------|
| Implementation       | Dedicated hardware devices | Software based (virtualized) |
| Cost                 | High                       | Low                          |
| Deployment Time      | Slow                       | Fast                         |
| Scalability          | Limited                    | High                         |
| Flexibility          | Low                        | High                         |
| Management           | Device-wise                | Centralized & Automated      |
| Resource Utilization | Low                        | High                         |
| Upgradation          | Hardware change required   | Software update (easy)       |

## 9. Conclusion

- NFV एक महत्वपूर्ण तकनीक है जो modern networks को अधिक flexible, scalable और cost-effective बनाती है। यह भविष्य के 5G, IoT और cloud based networks के लिए एक key enabler है।

# IoT and WoT

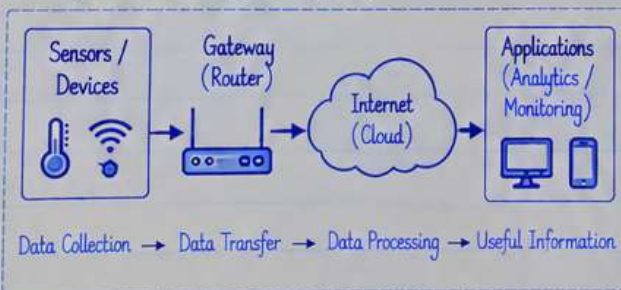
## 1. Introduction to IoT (Internet of Things)

- IoT (Internet of Things) एक ऐसा network है जिसमें physical devices, sensors, software और other objects internet के माध्यम से connect होते हैं और data collect, exchange और analyze करते हैं।
- IoT का उद्देश्य everyday devices को smart बनाना और उन्हें internet के जरिए communicate करने में सक्षम करना है।
- IoT में devices sensors के द्वारा real-time data प्राप्त करते हैं और उसे cloud या server पर भेजते हैं ताकि useful information प्राप्त हो सके।

## 2. Key Features of IoT

- ① Connectivity - devices का internet से connect होना।
- ② Sensing - environment से data collect करना।
- ③ Intelligence - data को process करके useful information बनाना।
- ④ Automation - tasks को automatically perform करना।
- ⑤ Real-time Monitoring - live data access और control।
- ⑥ Scalability - large number of devices को support करना।
- ⑦ Heterogeneous Devices - अलग-अलग प्रकार के devices का integration।

## 3. IoT Working Diagram



## 4. Applications of IoT

- Smart Home - home appliances का remote control
- Smart City - traffic, street lights, waste management
- Healthcare - patient monitoring devices
- Agriculture - soil moisture, weather monitoring
- Industrial IoT (IIoT) - machine monitoring, predictive maintenance
- Smart Wearables - fitness bands, smart watches
- Environmental Monitoring - air quality, water quality

**Note :** IoT physical world को digital world से connect करता है और data के माध्यम से smart और efficient solutions प्रदान करता है।

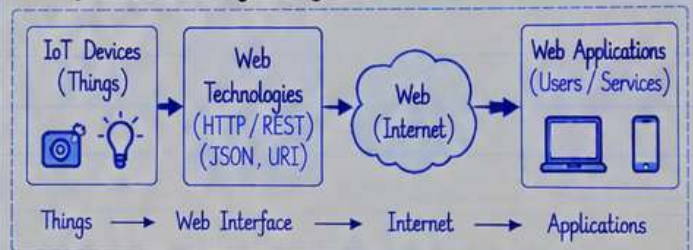
## 5. Introduction to WoT (Web of Things)

- WoT (Web of Things) IoT का एक extension है जिसमें web technologies का उपयोग करके things (devices) को web के साथ integrate किया जाता है।
- WoT का उद्देश्य IoT devices को standard web platforms द्वारा accessible बनाना है ताकि वे आसानी से web applications और services के साथ communicate कर सकें।
- यह REST APIs, HTTP, JSON जैसी web technologies का उपयोग करता है जिससे IoT devices को web की सभी सुविधाएँ प्राप्त होती हैं।

## 6. Key Features of WoT

- ① Uses Web Technologies - HTTP, REST, JSON, URI
- ② Interoperability - different devices और platforms के बीच better communication
- ③ Accessibility - devices को web के माध्यम से कहीं से भी access किया जा सकता है।
- ④ Standardization - open standards का उपयोग
- ⑤ Integration - IoT devices को existing web services और applications के साथ integrate करना।
- ⑥ Resource Sharing - data और services को web के माध्यम से share करना।

## 7. WoT Working Diagram



## 8. Difference between IoT and WoT

| Aspect          | IoT (Internet of Things)                            | WoT (Web of Things)                                      |
|-----------------|-----------------------------------------------------|----------------------------------------------------------|
| Meaning         | Network of physical devices connected via internet. | Uses web technologies to integrate IoT devices with web. |
| Technology      | Various (MQTT, CoAP, etc.)                          | Web technologies (HTTP, REST, JSON, URI)                 |
| Communication   | Device-to-device / device-to-cloud                  | Device-to-web (web-based communication)                  |
| Accessibility   | Limited (specific platforms)                        | Highly accessible via web browsers and APIs              |
| Standardization | Different protocols and proprietary systems         | Uses standard web protocols                              |
| Scope           | Focus on connecting things                          | Extends IoT with web integration                         |
| Example         | Smart sensors, smart home devices                   | Web-enabled IoT devices, web dashboards                  |

## 9. Conclusion

- IoT और WoT दोनों मिलकर एक smart, connected और intelligent world बनाने में मदद करते हैं।
- जहाँ IoT devices को connect करता है, वहीं WoT उन्हें web के साथ seamless integration प्रदान करता है।